

Heaven's Light Is Our Guide



Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

A Project Report on

**PIXAI - Political Image Cross Attention Inference:
A Multimodal Framework for Code-Mixed Bangla
Political Meme Classification**

Course Code: CSE 3200

Course Title: Software Development Project II

Author

Name: Tamjid Hasan Fahim

Roll: 2103052

Section: A

Supervisor

Utsha Das

Assistant Professor

Dept. of CSE, RUET

Date: January 8, 2026

Certification

This is to certify that the project titled: “**PIXAI - Political Image Cross Attention Inference: A Multimodal Framework for Code-Mixed Bangla Political Meme Classification**” submitted by **Tamjid Hasan Fahim**, Roll Number **2103052**, Section **A**, for the fulfillment of the requirements of the course **CSE 3200 (Software Development Project II)**, has been examined and approved by the undersigned.

The project has been carried out under my supervision, and it is recommended for submission and evaluation.

Supervisor:

Name: Utsha Das

Designation: Assistant Professor

Department: Computer Science & Engineering

Institution: Rajshahi University of Engineering & Technology (RUET)

Date: _____

Signature: _____

Abstract

Internet memes have evolved from ephemeral humor into sophisticated vehicles for political discourse, yet automated classification remains a significant challenge for low-resource, code-mixed languages due to the semantic gap between modalities and the failure of traditional OCR systems on noisy, mixed-script data. This project addresses these challenges by developing PIXAI (Political Image Cross Attention Inference), a novel multimodal framework for the binary classification of *Political* versus *Non-Political* memes in code-mixed Bangla (*Banglish*). The system introduces a Vision-Language Model based text extraction pipeline, shifting from conventional optical pattern recognition to semantic understanding of image-texts, enabling robust transcription of distorted and mixed-script text. The core architecture employs a dual-stream neural network that fuses frozen visual embeddings from CLIP with multilingual textual embeddings from XGLM using a dual Cross-Attention mechanism, allowing visual embeddings to query textual features and also attend jointly to both visual and textual embeddings for richer multimodal interactions. Trained on a curated dataset of 2,860 samples, the PIXAI model achieved an accuracy of 93.24% and a Macro F1-score of 0.9159, with a high precision of 94.59% for *political* content. The final system is deployed as a production-ready, asynchronous web application using FastAPI, optimized with 4-bit quantization for real-time inference on consumer-grade hardware.

Contents

Abstract	ii
1 Introduction	1
1.1 Problem Statement	1
1.2 Motivation	2
1.3 Objectives	2
1.4 Contribution Summary	3
1.5 Report Structure	4
2 Background & Literature Review	5
2.1 Theoretical Foundation	5
2.1.1 The Paradigm of Multimodal Representation Learning	5
2.1.1.1 Contrastive Language–Image Pre-training (CLIP)	5
2.1.1.2 Multilingual Autoregressive Modeling (XGLM)	6
2.1.2 The Mechanics of Cross-Modal Attention	7
2.1.3 Theoretical Shift in OCR: From Pattern Recognition to Vision–Language Understanding	7
2.1.4 Code-Mixing and Linguistic Entropy	8
2.2 Related Work Review	8
2.2.1 Global Trends in Multimodal Meme Analysis	8
2.2.2 Bengali and Code-Mixed Meme Research	8
2.2.3 Political vs. Non-Political Classification	8
2.2.4 Challenges in OCR and Text Extraction	9
2.3 Comparative Analysis	9
2.3.1 Architectural Comparison	9
2.3.2 Fusion Strategies	9
2.3.3 Text Extraction Techniques	9
2.3.4 Language Modeling for Code-Mixing	10
2.4 Gap Analysis	10
3 Requirements & Workflow	12
3.1 Functional Requirements	12
3.2 Non-functional Requirements	12
3.3 System Constraints	13

3.4	Technology Stack Justification	13
3.5	Team Structure & Individual Responsibilities	14
4	Project Management & Finance	15
4.1	Work Breakdown Structure (WBS)	15
4.2	Project Schedule — Gantt Chart	16
4.3	Budget & Financial Cost Analysis	17
4.3.1	Development & Infrastructure Costs	17
4.3.2	Operational Cost Projection (Monthly)	17
4.3.3	Cost Optimization Strategies	18
4.4	Risk Analysis & Mitigation Plan	18
4.5	Summary of Management Decisions	19
5	System Design & Architecture	20
5.1	Dataset Overview	20
5.2	Methodological Structure	21
5.2.1	Multimodal Data Enrichment through VLM Extraction	21
5.2.2	Textual Normalization and Noise Reduction	22
5.2.3	The PIXAI Architecture	23
5.2.3.1	Feature Extraction Streams	23
5.2.3.2	The Attention Mechanism	23
5.2.3.3	Fusion and Classification	23
5.2.4	Deployment	24
5.3	Deployment Module Interaction Diagram	24
5.4	Data Schema	26
5.5	API Contracts	26
5.6	Security & Performance Design	27
6	Implementation	28
6.1	Development Environment Setup	28
6.1.1	Model Training Environment (Research Phase)	28
6.1.2	Deployment Environment (Application Phase)	28
6.2	Software Configuration Management	28
6.2.1	Coding Standards and Formatting	28
6.2.2	Version Control Strategy	29
6.3	Model Training Hyperparameter	29
6.4	Neural Network Layer Details	30
6.5	Module-Wise Implementation	31
6.5.1	VLM Extraction Module	31
6.5.2	Text Cleaning and Normalization Pipeline	33
6.5.3	PIXAI Model Training Module	35
6.5.4	FastAPI Backend Implementation	37
6.5.4.1	Data Model for API Endpoint	38

6.5.4.2	Singleton Design Pattern Conception	38
6.5.4.3	Asynchronous Inference	39
6.6	Code Ownership Block	40
6.7	System Integration Workflow	41
6.8	Performance, Optimization, and Security Techniques	41
7	Result, Evaluation and Testing	43
7.1	Performance Evaluation	43
7.1.1	Research Results	43
7.1.1.1	Quantitative Metrics	43
7.1.1.2	Confusion Matrix Analysis	44
7.1.2	Deployed System Performance Metrics	44
7.2	Comparison with Baseline & Existing Systems	45
7.2.1	Comparison of Text Extraction Approaches	45
7.2.2	Comparison of Classification Architectures	45
7.3	Testing Strategy	46
7.3.1	Unit Testing	46
7.3.2	Integration Testing	46
7.3.3	System Testing	46
7.4	Test Case Tables	47
7.5	Screenshots of the Developed System	47
8	Discussion & Analysis	53
8.1	Interpretation of Findings	53
8.2	Strengths of the Project	53
8.3	Limitations	54
8.4	Recommendations for Future Development	55
8.5	Reflection on Design & Implementation Decisions	55
9	Life-Long Learning Impact	57
9.1	Technical & Professional Skills Acquired	57
9.2	Learning New Technologies & Tools	57
9.3	Future Growth & Directions	58
10	Conclusion	59
10.1	Summary of Work	59
10.2	Achievement of Objectives	59
10.3	Final Remarks	60
	References	60
A	Training Dynamics Log	64

List of Figures

4.1	Project Workflow Grantt Chart	16
5.1	Bar chart showing the label distribution of <i>Political</i> vs <i>Non-Political</i> memes.	21
5.2	Overall system architecture of PIXAI	22
5.3	Module Interaction Diagram	25
6.1	Neural Network Architecture Diagram	30
6.2	QR Code to the PIXAI Github Repository	40
7.1	Confusion Matrix for the PIXAI Meme Classification Model	44
7.2	Model Result Metrics Screenshot on Notebook	48
7.3	Webapp Interface	48
7.4	Correctly Classified Political Meme	49
7.5	A Correctly Classified Non-political Meme	50
7.6	A Misclassified Political Meme due to Contextual Knowledge Limitation of the Trained Model. See Section 8.1 for detailed discussion.	51
7.7	Server Initialization Log	52
7.8	Server Log During Inference	52
A.1	(Training vs Validation) Loss, Accuracy and Macro F1	64

List of Tables

4.1	Development and Infrastructure Cost Breakdown	17
5.1	Dataset Partitioning and Sample Counts	20
5.2	Label Distribution of <i>Political</i> vs <i>Non-Political</i> Memes	20
6.1	Training Hyperparameters for PIXAI	29
7.1	Classification Metrics for Political vs. Non-Political Content	43
7.2	Test Cases for PIXAI System	47
A.1	Training and Validation Metrics Across 5 Epochs	64

Listings

6.1	VLM Initialization and Prompt Engineering	31
6.2	Inference Generation Parameters	32
6.3	Noise Keyword Filtering Logic	33
6.4	Character Mapping for Stylized Unicode	34
6.5	Full Text Normalization Function	34
6.6	PIXAI Architecture and Attention Mechanism	36
6.7	Optimization and Loss Configuration	37
6.8	Prediction Response Data Model	38
6.9	Model and Service Initialization Using Lifespan Context	38
6.10	Main Inference Pipeline for Meme Classification	39

Chapter 1

Introduction

1.1 Problem Statement

In the contemporary landscape of digital communication, internet memes have transcended their origins as ephemeral humor to become sophisticated, high-impact vehicles for political discourse, propaganda, and ideological polarization. Unlike traditional unimodal content (plain text or standalone imagery), memes function as *multimodal units of cultural transmission*, where meaning is constructed not through the sum of parts, but through the complex, often non-linear interaction between visual cues (templates, facial expressions, iconography) and embedded textual overlays (captions, punchlines).

The central problem addressed by this research is the automated binary classification of memes into “Political” or “Non-Political” categories, a task that remains computationally widely unresolved for low-resource and code-mixed languages. Current state-of-the-art systems face three critical failure modes:

1. **The Semantic Gap in Multimodality:** Unimodal classifiers fail to capture the *intersemiotic complementarity* of memes. A seemingly benign image (e.g., a popular movie scene) can become highly politically charged when overlaid with specific text, and conversely, aggressive text may be neutralized by a humorous visual context. Standard models that analyze text and image in isolation miss this implicit reasoning.
2. **Linguistic Complexity (Code-Mixing):** In the South Asian context, specifically Bangladesh, memes frequently exhibit *code-switching*—the linguistic phenomenon of alternating between Bangla (in native script) and English within a single instance. Traditional Optical Character Recognition (OCR) engines struggle to transcribe this mixed-script data accurately, often producing “gibberish” outputs that degrade downstream classification performance.
3. **Visual Noise and Distortion:** Memes are often characterized by high visual noise, including compression artifacts, watermarks, and non-standard fonts. This visual entropy renders standard feature extraction techniques ineffective, necessitating advanced Vision-Language Models (VLMs) capable of “reading” text in the wild.

This project seeks to resolve these challenges by engineering an end-to-end pipeline that integrates granular VLM-based extraction with a specialized dual-stream cross-attention network (PIXAI) to decipher the latent political intent of code-mixed memes.

1.2 Motivation

The motivation for this research is situated at the intersection of Social Media Forensics, Multimodal Artificial Intelligence, and Computational Sociolinguistics.

Societal Imperative Social media platforms have been effectively “weaponized” for geopolitical events, where memes serve as tools for subversion, disinformation, and the reinforcement of echo chambers. The sheer volume of user-generated content makes manual moderation impossible. An automated system capable of distinguishing political satire or propaganda from harmless entertainment is essential for:

- **Content Moderation at Scale:** Filtering potential disinformation campaigns or hate speech disguised as humor.
- **Echo Chamber Analysis:** Understanding how political narratives propagate through homogenous user groups.
- **Preserving Democratic Discourse:** Identifying content that seeks to polarize public opinion while protecting legitimate political expression.

Technical Research Gap While significant progress has been made in analyzing English-centric memes, there is a distinct scarcity of research focusing on Bengali-English code-mixed content. Existing multimodal models often rely on simple concatenation of features (Early or Late Fusion) which fails to model the *interaction* between modalities. This research is motivated by the need to move beyond “black-box” classification toward an architecture (PIXAI) that explicitly models the reasoning process—using the image to “query” the text for relevant semantic signals. Furthermore, leveraging the emergent capabilities of Large Vision-Language Models (like Qwen2.5-VL) for data cleaning represents a novel approach to overcoming the limitations of traditional OCR in noisy environments.

1.3 Objectives

The overarching goal of this study is to develop a high-precision, multimodal deep learning framework for the classification of political memes. The project is driven by the following measurable technical objectives:

- **Objective 1: Granular Multimodal Extraction**
To develop and optimize Vision-Language Model (VLM) based meme-caption extraction. The system aims to achieve high-fidelity transcription of code-mixed (Bangla/English) text while semantically distinguishing between relevant meme captions and irrelevant background noise (watermarks, UI elements).

- **Objective 2: Robust Textual Normalization**

To engineer a comprehensive cleaning pipeline that addresses the linguistic entropy of internet text. This involves implementing NFKC Unicode normalization, artifact removal (e.g., emojis, URLs), and standardization of mixed-script tokens to ensure consistent input for the language model.

- **Objective 3: Development of the PIXAI Architecture**

To design and train a novel multimodal neural network, PIXAI (Political Image Cross Attention & Inference). This objective focuses on implementing a Cross-Attention mechanism that fuses static visual embedding with dynamic textual embeddings, thereby modeling the semantic alignment between the visual and textual domains.

- **Objective 4: Real-Time Inference Deployment**

To integrate the trained models into a production-ready web API. This objective aims to provide a scalable interface where end-users can upload raw images and receive instantaneous classification predictions (Political vs. Non-Political) along with confidence probabilities.

1.4 Contribution Summary

This research makes several distinct contributions to the fields of Multimodal Deep Learning and Computational Sociolinguistics, particularly in the context of low-resource, code-mixed Bengali-English content.

Development of the PIXAI Architecture: This work introduces PIXAI (Political Image Cross Attention Inference), a novel dual-stream neural architecture. Unlike traditional multimodal models that rely on early or late feature fusion, PIXAI employs a dual cross-attention mechanism. In the first step, visual embeddings generated by CLIP dynamically query textual embeddings produced by XGLM, explicitly modeling semantic alignment between image and text. In a complementary hybrid attention step, visual embeddings attend jointly to both textual and visual features, enabling richer multimodal interactions. This architecture preserves modality-specific representations while allowing relevance-driven fusion, improving the model’s ability to infer nuanced political intent in memes.

Novel VLM-Based Text Extraction Pipeline: To overcome the limitations of traditional OCR systems in handling noisy, mixed-script “Banglish” text, this project implements a Vision-Language Model (VLM)-based extraction pipeline using Qwen/Gemini. This approach transitions from optical pattern recognition to semantic scene understanding, enabling accurate transcription of code-mixed internet text without transliteration artifacts.

Dedicated Political vs. Non-Political Classification: Unlike prior Bengali meme analysis research that often conflates political discourse with hate speech detection, this work establishes a dedicated binary political classification baseline. This distinction allows for the analysis of political narratives that may be satirical or neutral without being hateful.

Production-Ready Asynchronous Deployment: Beyond theoretical modeling, this project delivers a real-time, production-ready inference system. The complete multimodal

pipeline is integrated into an asynchronous FastAPI backend and optimized using 4-bit quantization, allowing efficient deployment on consumer-grade hardware while supporting concurrent user requests.

1.5 Report Structure

The remainder of this report is organized as follows:

Chapter 2: Background & Literature Review establishes the theoretical foundation of the project by discussing CLIP, XGLM, and cross-modal attention mechanisms. It also reviews existing literature on meme analysis and identifies research gaps in Bengali code-mixed content.

Chapter 3: Requirements & Team Workflow outlines the functional and non-functional system requirements, justifies the selected technology stack (FastAPI, PyTorch, Gemini/Qwen), and describes the team structure and collaboration methodology.

Chapter 4: Project Management & Finance presents the Work Breakdown Structure (WBS), project timeline using a Gantt chart, budget analysis, and risk mitigation strategies applied during development.

Chapter 5: System Design & Architecture provides a detailed technical overview of the PIXAI system, including the four-phase methodology ranging from VLM-based text extraction and normalization to cross-attention-based fusion. Architectural and sequence diagrams are also included.

Chapter 6: Implementation documents the development environment, coding standards, and module-wise implementation details. It explains the algorithms used for both the VLM extraction module and the PIXAI classification engine, supported by relevant code snippets.

Chapter 7: Testing, Results & Evaluation describes the testing strategies employed (unit, integration, and system testing) and presents quantitative performance metrics such as confusion matrices, F1-scores, and system latency.

Chapter 8: Discussion & Analysis interprets the experimental results, highlighting the model’s strengths in handling code-mixed data and discussing its limitations in capturing subtle satire. Potential future enhancements, including edge-device deployment, are also proposed.

Chapter 9: Life-Long Learning Impact reflects on the technical and professional skills acquired during the project, including multimodal system design, prompt engineering, and asynchronous web development.

Chapter 10: Conclusion summarizes the overall work, confirms the achievement of the initial research objectives, and provides final remarks on the project’s significance in the field of automated content moderation.

Chapter 2

Background & Literature Review

2.1 Theoretical Foundation

The development of a robust, automated system for the binary classification of political memes—specifically within the linguistically complex environment of code-mixed Bengali and English—necessitates a rigorous theoretical grounding. This research operates at the convergence of three distinct but increasingly intertwined computational domains: *Multi-modal Deep Learning*, *Computational Sociolinguistics*, and *Vision–Language Understanding*. To effectively engineer the proposed PIXAI (Political Image Cross Attention & Inference) architecture and the associated VLM-based extraction pipeline, it is imperative to deconstruct the theoretical underpinnings of Contrastive Language–Image Pre-training (CLIP), Multilingual Autoregressive Language Modeling (XGLM), and the intricate mechanics of Cross-Modal Attention. Furthermore, the methodological transition from traditional Optical Character Recognition (OCR) to Vision–Language Model (VLM) extraction represents a significant paradigm shift in the processing of noisy, unstructured data that warrants a comprehensive theoretical examination.

2.1.1 The Paradigm of Multimodal Representation Learning

Multimodal learning addresses the fundamental computational challenge of synthesizing information from multiple sensory modalities—in this context, vision and language—to perform complex reasoning tasks. The core theoretical obstacle in this domain is the *semantic gap*, a phenomenon where the low-level pixel representation of an image and the high-level semantic representation of text do not naturally align in a shared vector space. A pixel grid containing a politician’s face and a text string containing a satirical caption exist in disparate feature spaces; bridging this gap requires a unified embedding manifold where geometrically proximate vectors correspond to semantically aligned concepts.

2.1.1.1 Contrastive Language–Image Pre-training (CLIP)

The visual encoding backbone of the proposed system relies heavily on the theoretical principles established by CLIP. Unlike traditional computer vision models (such as ResNet [1] or EfficientNet[2]), which are trained on fixed sets of discrete labels (e.g., the 1,000 classes

of ImageNet), CLIP is trained on a massive, uncuration corpus of 400 million image–text pairs collected from the internet.

The fundamental theoretical innovation of CLIP is its departure from predictive classification objectives (predicting a label y given an image x) toward a contrastive loss objective grounded in self-supervised learning. Given a batch of N image–text pairs, CLIP trains two parallel encoders—an image encoder (typically a Vision Transformer or ResNet) and a text encoder (a Transformer)—to predict which of the $N \times N$ possible pairings across the batch actually occurred. The model seeks to maximize the cosine similarity of the image and text embeddings for the N real pairs (the diagonal of the similarity matrix) while simultaneously minimizing the cosine similarity for the $N^2 - N$ incorrect pairings.

The optimization objective uses a symmetric cross-entropy loss function, often referred to as the InfoNCE loss, defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(I_i, T_i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(I_i, T_j)/\tau)}$$

where I_i and T_i represent the normalized feature embeddings of the image and text, respectively, and τ is a learnable temperature parameter that scales the logits to control the sharpness of the probability distribution.

The significance of this framework for political meme classification is profound. It bestows the model with *zero-shot* transfer capabilities. Because CLIP learns visual concepts directly from natural language supervision rather than categorical indices, it can recognize visual patterns—such as a specific politician’s face, a protest symbol, or a flag—without explicit task-specific training. However, literature indicates a critical limitation: while CLIP excels at general visual recognition and high-level semantic alignment, it often struggles with fine-grained text recognition (OCR) inside images and the culture-specific reasoning required for abstract concepts such as irony or sarcasm in low-resource languages. This necessitates coupling CLIP with specialized multilingual text encoders and sophisticated attention mechanisms for Bengali meme classification.

2.1.1.2 Multilingual Autoregressive Modeling (XGLM)

For the textual stream, particularly in handling the linguistic entropy of code-mixed Bangla–English (“Banglish”), the theoretical framework shifts from masked language modeling to autoregressive modeling. XGLM (Cross-lingual Generative Language Model) is a decoder-only transformer trained on a balanced corpus covering a diverse set of languages, including Bengali.

Unlike encoder-only models (e.g., XLM-R [3]), which are optimized for discriminative tasks via masked token prediction, XGLM is trained using a causal language modeling objective. Its goal is to maximize the probability of the next token x_t given all preceding tokens $x_{1:t-1}$. The probability of a sequence x is defined by the chain rule:

$$P(x) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

Theoretical comparisons suggest that while encoder-only models often perform better on classification tasks due to bidirectional context visibility, autoregressive models like XGLM demonstrate superior few-shot learning capabilities and robustness to syntactic irregularities inherent in code-switching. Code-mixing often results in fragmented or non-standard grammatical constructions, which autoregressive models can model more flexibly. Additionally, the curse of multilinguality is mitigated in XGLM through massive scaling (up to 7.5 billion parameters) and balanced sampling strategies, making it particularly robust for South Asian linguistic data.

2.1.2 The Mechanics of Cross-Modal Attention

The central architectural innovation of the PIXAI system is its move beyond early fusion or late fusion toward intermediate fusion via Cross-Attention [4]. Cross-Attention generalizes the Transformer’s self-attention mechanism to fuse heterogeneous data streams dynamically.

In self-attention, the Query (Q), Key (K), and Value (V) matrices are derived from the same input. In Cross-Attention, these components originate from different modalities. For the visual-query configuration employed in PIXAI, the operation is defined as:

$$\text{Attention}(Q_v, K_t, V_t) = \text{softmax} \left(\frac{Q_v K_t^\top}{\sqrt{d_k}} \right) V_t$$

where Q_v represents visual features from CLIP, K_t and V_t represent textual features from XGLM, and d_k is the key dimensionality used for scaling.

This formulation models the cognitive process of “looking for evidence.” In the first attention step, visual features query textual embeddings to identify semantically relevant tokens, with high dot-product similarity producing stronger attention weights that guide information aggregation. In a complementary hybrid attention step, visual features attend jointly to both visual and textual embeddings, capturing richer cross-modal interactions. Unlike simple concatenation, this dual cross-attention preserves modality-specific representations while enabling relevance-driven fusion, allowing the model to focus on aligned cues that define political intent in memes.

2.1.3 Theoretical Shift in OCR: From Pattern Recognition to Vision–Language Understanding

This project employs a Vision–Language Model (Qwen2.5-VL) for text extraction, marking a departure from traditional OCR systems such as Tesseract. Conventional OCR relies on sequential pipelines—binarization, segmentation, and character classification—assuming clean, linear, high-contrast documents. These assumptions fail catastrophically for memes, which exhibit high visual entropy, including multicolored backgrounds, distortions, stylized fonts, and overlaid text.

In contrast, VLM-based extraction frames OCR as a sequence-to-sequence generation task conditioned on visual input. Qwen2.5-VL uses a Vision Transformer with dynamic resolution handling and a language decoder that generates text token by token. Theoretical

advantages arise from semantic priors: the model leverages language probability to infer obscured characters using contextual cues rather than pixel shape alone.

This capability is crucial for Bengali memes, where traditional OCR engines often fail due to complex ligatures (*juktakkhor*) and mixed-script text. VLMs “read by understanding,” dramatically improving robustness under noise.

2.1.4 Code-Mixing and Linguistic Entropy

Code-mixing is a structured linguistic phenomenon governed by constraints such as the Matrix Language Frame model. In Bengali memes, this appears as Banglish or intra-sentential script switching. Monolingual NLP models suffer distributional shift when processing such data, often fragmenting mixed words into meaningless subunits.

XGLM’s multilingual pre-training allows it to approximate the interlanguage space of code-mixed text more effectively. Additionally, NFKC normalization in preprocessing ensures Unicode canonicalization, reducing sparsity caused by visually identical but byte-wise distinct characters.

2.2 Related Work Review

2.2.1 Global Trends in Multimodal Meme Analysis

The release of the Hateful Memes Challenge dataset [5] by Facebook AI in 2020 marked a turning point. The dataset introduced benign confounders—memes where identical images convey different meanings depending on text—forcing models to learn multimodal relationships rather than unimodal shortcuts. Subsequent tasks, such as SemEval-2024 Task 4 [6], expanded the scope to multilingual persuasion technique detection, revealing persistent challenges in cultural reasoning and non-English contexts.

Other datasets like HarMeme introduced target identification, emphasizing intent-aware and persuasion-aware modeling beyond binary hate detection.

2.2.2 Bengali and Code-Mixed Meme Research

Research in Bengali meme analysis is emerging. The MUTE [7] dataset established the baseline for multimodal Bengali meme classification, demonstrating performance gains over unimodal models. BanglaAbuseMeme [8] confirmed the superiority of multimodal approaches while highlighting difficulties in code-mixed embeddings. The MIMOSA dataset [9] explicitly separated political aggression, proposing attention-based fusion models that outperform concatenation. Other works such as MemoSen exposed shortcut learning tendencies, motivating architectures like PIXAI.

2.2.3 Political vs. Non-Political Classification

Binary political classification remains underexplored. Studies on US political memes showed that visual style is a strong predictor of political intent. Research on fringe communities

demonstrated different propagation dynamics for political memes. Crucially, political content is not inherently hateful, and existing Bengali datasets conflate political discourse with aggression, leaving a gap for neutral political classification.

2.2.4 Challenges in OCR and Text Extraction

Traditional OCR systems are a bottleneck in meme analysis. Studies consistently show that OCR failures propagate errors downstream. Bengali script complexity exacerbates this issue. Recent work advocates VLM-based scene text recognition, which leverages semantic reasoning to transcribe noisy, code-mixed text accurately.

2.3 Comparative Analysis

2.3.1 Architectural Comparison

Text-only models, such as mBERT [10] and BanglaBERT [11], focus primarily on keyword detection and are effective at capturing textual cues. However, they are often misled by benign confounders and struggle to handle content where context is derived from both text and visuals. Image-only models, including ResNet and ViT, excel at visual entity recognition and capturing spatial features, but their performance deteriorates when processing content that relies heavily on textual information, such as memes containing embedded text. Multimodal architectures that integrate both visual and textual representations, often through mechanisms like cross-attention, demonstrate improved understanding by leveraging intermodal complementarities. Despite these advantages, such models typically demand higher computational resources and can be more complex to train effectively.

2.3.2 Fusion Strategies

Various fusion strategies have been proposed to combine information from multiple modalities. Simple concatenation merges image and text feature vectors into a single representation, providing a straightforward approach but lacking the ability to focus selectively on relevant information. Attention-based fusion methods, particularly cross-attention, enable dynamic alignment between modalities, allowing the model to emphasize the most informative visual regions and textual tokens. While this approach enhances representational power, it introduces additional computational complexity and requires careful design to prevent overfitting or misalignment.

2.3.3 Text Extraction Techniques

Text extraction from images is another critical component for content understanding. Traditional OCR methods, such as Tesseract, rely on character recognition and often produce unstructured outputs. They are sensitive to noise and perform poorly on code-mixed or stylized text, limiting their effectiveness in diverse, real-world scenarios. In contrast, vision-language-based extraction techniques utilize semantic understanding to generate structured outputs, handle noise more robustly, and support code-mixed content. Nev-

ertheless, these methods may require large-scale training data and can be computationally intensive.

2.3.4 Language Modeling for Code-Mixing

Handling informal or code-mixed text presents significant challenges for language modeling. Encoder-only models, such as BERT-based architectures, provide bidirectional context and are generally effective at capturing fragmented or non-standard text, although they can be limited when confronted with highly unconventional language patterns. Decoder-only autoregressive models, such as XGLM, are capable of modeling sequential dependencies and handling high-entropy, multilingual content due to their large-scale training. While these models can be adapted for tasks such as classification through text-to-label generation or embedding extraction, their performance depends heavily on the diversity and quality of the pretraining data, and they may be less efficient at capturing bidirectional contextual nuances compared to encoder-based approaches.

2.4 Gap Analysis

Despite significant advances in multimodal and text analysis for Bengali content, several limitations persist in existing approaches. These gaps hinder accurate understanding of complex memes and political content, particularly in the presence of code-mixing, noisy text, and cross-modal interactions. The key research gaps can be summarized as follows:

1. **Fusion Fidelity Gap:** Most existing Bengali models rely on simple fusion strategies, which limit the ability to capture nuanced interactions between modalities. Cross-attention mechanisms enable more interpretable and cognitively grounded multimodal interactions.
2. **Political Classification Gap:** Prior work often conflates political content with hate speech, which can bias downstream analysis. Dedicated classification of political versus non-political content allows for more precise discourse understanding.
3. **OCR Noise Gap:** Traditional OCR techniques struggle with code-mixed or stylized meme text, leading to noisy or unstructured outputs. Vision-language-based extraction methods provide a shift from purely optical recognition to semantic understanding.
4. **Semantic Gap in Unimodal Baselines:** Unimodal models fail to capture inter-semiotic meaning and cross-modal dependencies. Leveraging multimodal features with attention-based fusion helps bridge this semantic gap.

Summary

The theoretical foundation of this project integrates advances in multimodal deep learning, generative language modeling, and vision-language understanding. The literature reveals

clear gaps in attention-based fusion, political classification, and robust text extraction for code-mixed Bengali memes. The proposed PIXAI architecture and VLM-based pipeline are positioned to address these gaps, advancing the state of the art from pattern recognition toward deep semantic interaction.

Chapter 3

Requirements & Workflow

3.1 Functional Requirements

The system serves as a web-based automated classification tool for meme content. From a user perspective, the system must perform the following functions:

1. **Image Upload & Validation:** Users must be able to upload meme images via a drag-and-drop interface or file selection. The system must validate that the uploaded file is a supported image format (JPEG, PNG, GIF) before processing.
2. **Automated Text Extraction:** The system must automatically detect and extract embedded text from the image, specifically handling mixed-script (Bangla and English) content and distinguishing it from background noise.
3. **Content Classification:** Upon analysis, the system must classify the meme into one of two binary categories: “Political” or “Non-Political”.
4. **Confidence Reporting:** The system must display the classification result along with a confidence score and probability breakdowns for both classes to ensure transparency.
5. **Transparency & Verification:** The system must display the raw “Extracted Text” and the “Normalized Text” to allow the user to verify what the model actually recognized.

3.2 Non-functional Requirements

1. **Performance:** The system is designed for real-time inference. The extraction module must use efficient models to minimize latency, while the backend should employ asynchronous processing (`async def`) to handle concurrent requests without blocking.
2. **Usability:** The interface is designed with a clean and responsive layout, providing immediate visual feedback (loading spinners, color-coded results) to guide users through the analysis process.
3. **Security:**

- (a) **Input Validation:** The API enforces strict content-type checking to prevent non-image file uploads.
 - (b) **Credential Management:** Sensitive keys (e.g., API keys) are stored in environment variables and not hardcoded, preventing accidental exposure.
4. **Robustness:** The text extraction module includes error handling and regex fallbacks to ensure the system does not crash if the VLM returns malformed or unexpected outputs.

3.3 System Constraints

- 1. **Hardware Constraints:** The PIXAI classification model requires a CUDA-enabled GPU for efficient inference during the training phase. Local VLM components are optimized to run within approximately 6GB of VRAM using 4-bit quantization.
- 2. **Network Dependency:** When running inference, an active internet connection is required to communicate with the Google Gemini API for text extraction.
- 3. **Software Environment:** The application relies on a Python environment with `torch==2.1.2`, `fastapi==0.109.0`, and `google-generativeai==0.3.2`, and is designed to run on Linux or Windows systems capable of supporting the NVIDIA CUDA toolkit.
- 4. **API Quotas:** System throughput is constrained by the rate limits and quotas associated with the external Google Gemini API service.

3.4 Technology Stack Justification

- **FastAPI:** Chosen for its high performance, native support for asynchronous programming (crucial for I/O bound ML tasks), and automatic generation of interactive API documentation.
- **Google Gemini (via Generative AI SDK):** Selected for text extraction because traditional OCR engines (e.g., Tesseract) fail on code-mixed “Banglish” memes. Gemini provides granular OCR capabilities and semantic understanding to distinguish caption text from background noise.
- **PyTorch:** Utilized as the primary deep learning framework for implementing the PIXAI architecture, offering flexible dynamic computation graphs necessary for cross-attention mechanisms.
- **CLIP (OpenAI) & XGLM:**
 - **CLIP:** Used for the visual stream to leverage zero-shot transfer capabilities and robust visual feature extraction.

- **XGLM:** A multilingual autoregressive language model, chosen for the textual stream to effectively handle the linguistic entropy and code-switching inherent in Bengali-English memes.

3.5 Team Structure & Individual Responsibilities

This project was carried out as an individual effort for the CSE 3200 course. As the sole developer, the author of this report assumed all roles and responsibilities associated with the project lifecycle, from initial conceptualization to final deployment and documentation.

Chapter 4

Project Management & Finance

4.1 Work Breakdown Structure (WBS)

The project is systematically divided into two major phases to clearly separate the scientific research component from the software engineering and deployment component. This separation ensures methodological rigor in model development while maintaining engineering robustness in system implementation.

Phase I: Research & PIXAI Model Development

1. Problem Formulation & Literature Review

- Studying existing multimodal fusion strategies, including early and late fusion approaches.
- Identifying research gaps in Bengali code-mixed meme classification.

2. Data Curation & Enrichment Pipeline

- Aggregating raw meme datasets labeled as Political and Non-Political.
- Developing the Vision–Language Model (VLM) extraction module for optical text extraction.
- Implementing a text normalization pipeline using Unicode normalization (NFKC) and regex-based cleaning to handle the noise.

3. PIXAI Architecture Training

- Designing the cross-attention mechanism using PyTorch.
- Training the model using the MADGRAD optimizer and a weighted loss function to address class imbalance.
- Validating model performance using standard metrics and saving trained checkpoints.

Phase II: Application Development & Inference Deployment

1. Backend API Engineering

- Setting up the FastAPI application structure.
- Integrating the trained PIXAI model for inference via a dedicated prediction endpoint.
- Implementing asynchronous handling for external Gemini API calls.

2. Frontend Interface Development

- Developing a web-based user interface with drag-and-drop image upload functionality.
- Visualizing classification confidence scores and probability distributions.

3. System Integration & Testing

- Conducting end-to-end testing across the full pipeline: image upload, text extraction, normalization, and classification.
- Managing deployment configuration and dependency control using `requirements.txt`.

4.2 Project Schedule — Gantt Chart

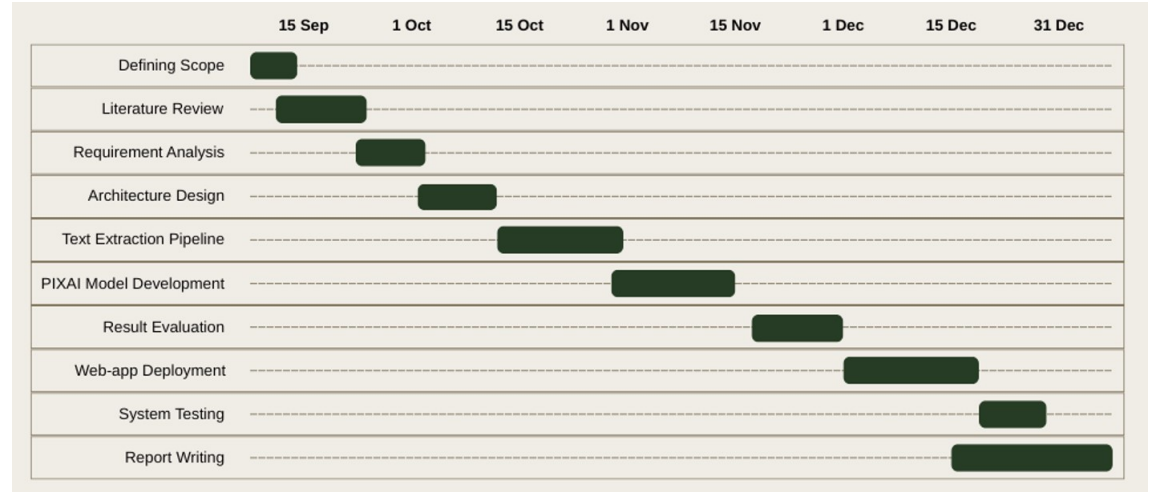


Figure 4.1: Project Workflow Grantt Chart

Figure 4.1 visually summarizes the project timeline. The project lifecycle was executed over a comprehensive 12-week period, commencing in mid-September 2025 and concluding in late December 2025. The schedule was structured into two distinct but overlapping phases: Phase I (Research & Model Development) and Phase II (System Development & Deployment).

The research phase focused on establishing the theoretical foundation, reviewing previous works on this doamin, curating the dataset, and developing, training and evaluating the

PIXAI architecture. The system development phase followed sequentially, emphasizing the integration of trained models into a production-ready asynchronous web application.

4.3 Budget & Financial Cost Analysis

This project was developed primarily using open-source technologies and educational tiers of cloud services, resulting in a minimal direct financial footprint. However, a commercial cost estimation was conducted to evaluate the economic feasibility of scaling the PIXAI system. The financial analysis is divided into Development Costs (Capital Expenditure) and Operational Costs (Operational Expenditure).

4.3.1 Development & Infrastructure Costs

The development phase adopted a hybrid computing strategy, utilizing Google Colab for model training and local hardware for application development and testing.

Item	Description	Actual Cost	Commercial Value
Compute Resources	Google Colab Pro (3 months) for model training using NVIDIA T4/A100 GPUs	\$0.00 (Free Tier Used)	\$30.00
Local Hardware	Depreciation of personal computer (16GB RAM, Intel i3500 CPU) used for development and testing	\$0.00 (Existing Asset)	\$25.00 (Estimated)
Software Stack	PyTorch, FastAPI, Hugging Face Transformers, Qwen VLM (open-source licenses)	\$0.00	\$0.00
API Credits	Google Gemini API for text extraction during inference	\$0.00 (developer free tier used)	\$20.00
Total Development Cost		\$0.00	\$75.00

Table 4.1: Development and Infrastructure Cost Breakdown

4.3.2 Operational Cost Projection (Monthly)

For a hypothetical production deployment handling approximately 10,000 requests per month, the estimated operational costs are outlined below. This projection assumes a transition from a local execution environment to a cloud-hosted infrastructure.

- **Cloud Hosting (Backend):** Deployment on an AWS EC2 instance (g4dn.xlarge with NVIDIA T4) or a CPU-only instance when using quantized models. Estimated cost: \$150 per month.
- **API Usage (Gemini):** Commercial usage costs based on token consumption. Assuming a rate of \$0.000125 per request for 10,000 monthly requests. Estimated cost: \$1.25–\$5.00 per month.

- **Storage:** Storage of model weights (approximately 3GB) and user logs using AWS S3. Estimated cost: \$2.00 per month.

4.3.3 Cost Optimization Strategies

To ensure financial sustainability and scalability, several cost optimization techniques were employed throughout the project:

1. **Model Quantization:** The use of 4-bit quantization (NF4) enabled the 7B-parameter vision-language model to run on GPUs with as little as 8GB VRAM, eliminating the need for expensive enterprise-grade hardware such as NVIDIA A100 GPUs.
2. **Asynchronous Processing:** Implementing an asynchronous FastAPI backend maximized throughput on a single server instance, significantly reducing the requirement for horizontal scaling.
3. **Frozen Encoders:** Freezing the CLIP visual encoder during training reduced back-propagation overhead, leading to shorter training times and lower computational and energy costs.

4.4 Risk Analysis & Mitigation Plan

Potential risks were identified across both the research and deployment phases of the project. Each identified risk was addressed with a corresponding mitigation strategy, as outlined below:

1. **Model Convergence Instability:** The cross-attention mechanism faced potential convergence issues due to the semantic mismatch between visual features extracted by CLIP and textual features generated by XGLM. This issue was mitigated by adopting the MADGRAD optimizer, which provides improved stability and convergence behavior for transformer-based architectures.
2. **Noisy or Hallucinated Text Extraction:** Vision-Language Models occasionally produced noisy or hallucinated text outputs for complex, code-mixed meme images. To address this, a robust text normalization module incorporating regex-based filtering and Unicode normalization was implemented to remove artifacts before classification.
3. **API-Induced Inference Latency:** Reliance on the external Google Gemini API for text extraction introduced network latency that could degrade user experience. This was mitigated by implementing asynchronous request handling using `async/await`, allowing the backend to process multiple user requests concurrently without blocking.
4. **Deployment Environment Reproducibility Issues:** The dependency on a specific PyTorch and CUDA configuration posed challenges for deployment across different systems. To mitigate this risk, all dependencies were frozen in `requirements.txt`, and a dedicated `/health` endpoint was introduced to verify successful model loading during system startup.

4.5 Summary of Management Decisions

Several key management decisions were taken throughout the project lifecycle to address technical challenges and ensure system robustness.

1. **Adoption of Vision–Language Models for Text Extraction:** Traditional OCR engines failed to handle code-mixed “Banglish” memes effectively. Vision–Language Models were adopted to treat text extraction as a semantic understanding problem rather than simple pattern recognition.
2. **Cross-Attention over Feature Concatenation:** Simple early fusion approaches failed to capture meaningful interactions between visual and textual modalities. A custom cross-attention mechanism was implemented to enable contextual reasoning across modalities.
3. **Asynchronous FastAPI Architecture:** Given the combination of network-bound and compute-bound operations in the inference pipeline, FastAPI was selected for its native asynchronous support, enabling efficient handling of concurrent user requests.

Chapter 5

System Design & Architecture

5.1 Dataset Overview

The dataset utilized for this research is a curated collection of code-mixed Bengali (Banglish) memes, labelled as *political* or *non-political*, sourced and aggregated from the *PoliMeme Decode* dataset [12]. The corpus consists of varying image resolutions and textual styles, representing a diverse range of political satire, commentary, and neutral content prevalent on social media platforms in Bangladesh.

The complete dataset comprises a total of 2,860 samples. To ensure robust model evaluation and prevent overfitting, we employed a stratified splitting strategy, partitioning the data into Training (70%), Validation (15%), and Testing (15%) sets. The specific distribution of samples across these partitions is detailed in Table 5.1.

Split Type	Percentage	Sample Count
Training Set	70%	2,002
Validation Set	15%	429
Test Set	15%	429
Total	100%	2,860

Table 5.1: Dataset Partitioning and Sample Counts

The dataset exhibits a natural class imbalance, reflecting real-world distributions where non-political content dominates social media memes. The label counts are as follows:

Label	Count
Non-Political	2,007
Political	853
Total	2,860

Table 5.2: Label Distribution of *Political* vs *Non-Political* Memes

This distribution is visualized in Figure 5.1, showing that non-political memes significantly outnumber political ones, emphasizing the challenge of detecting political content in a predominantly neutral environment. The dataset has been kept imbalanced during training to reflect the real-life prevalence of non-political content; however, a weighted loss

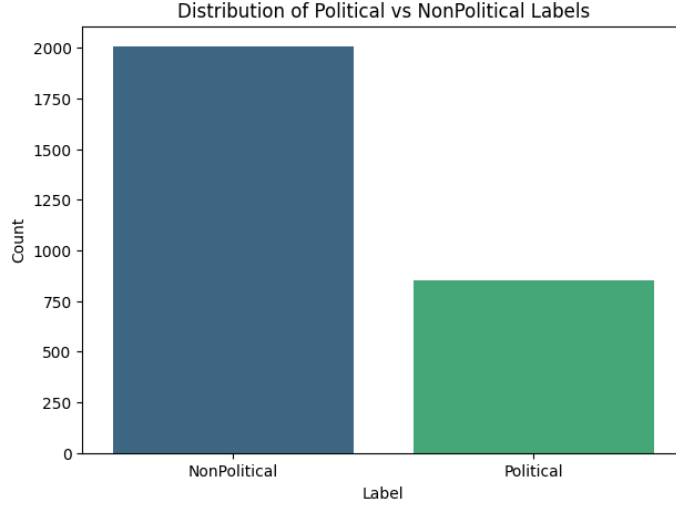


Figure 5.1: Bar chart showing the label distribution of *Political* vs *Non-Political* memes.

has been calculated to compensate for the class imbalance and ensure the model properly learns the minority *Political* class.

5.2 Methodological Structure

The end-to-end methodology of the proposed PIXAI system, illustrated in Figure 5.2, follows a structured lifecycle from raw data acquisition to real-world deployment, designed for multimodal image classification and understanding. The architecture balances efficient data preprocessing with sophisticated neural network design. Below is a breakdown of the system in distinct phases.

5.2.1 Multimodal Data Enrichment through VLM Extraction

Meme analysis presents a unique challenge where the embedded text is often optically distorted, stylized, or mixed with code-switching (e.g., Bangla and English). Traditional OCR engines often fail to capture the semantic context of the scene. To address this, we utilize Qwen2.5-VL-7B-Instruct, [13, 14, 15]. a state-of-the-art tiny yet powerful 7 billion parameter Vision-Language Model.

- **Quantization & Optimization:** To ensure computational feasibility within resource-constrained environments (e.g., consumer GPUs), the model is deployed with 4-bit quantization using the `bitsandbytes` library (NF4 format). This reduces the memory footprint significantly while maintaining inference fidelity.
- **Prompt Engineering:** We employ a structured prompting strategy that instructs the VLM to perform a granular scan of the image. The model is explicitly directed to:

1. Differentiate between caption text and background noise (watermarks, UI elements).

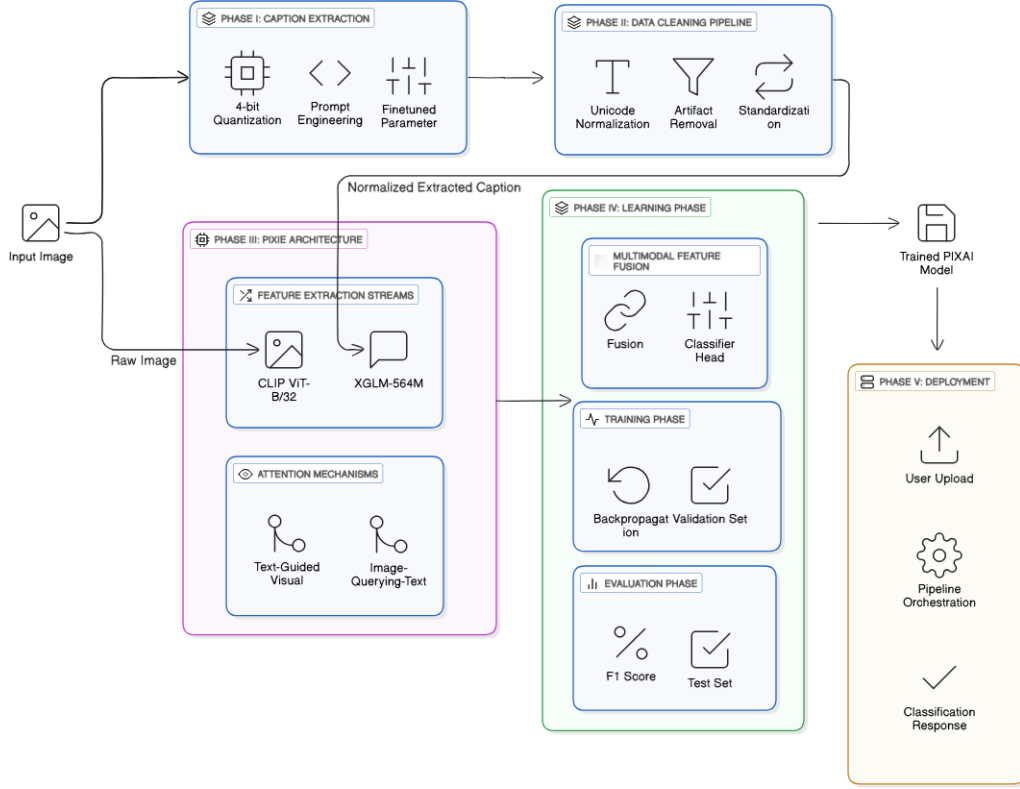


Figure 5.2: Overall system architecture of PIXAI

2. Handle mixed-script (Bangla/English) transcription without transliteration.
3. Output the data in a structured JSON format containing a list of **detected_text** blocks.

- **Preprocessing:** Input images are dynamically resized to a maximum dimension of 1024 pixels to balance token usage and visual clarity before being passed to the VLM.

5.2.2 Textual Normalization and Noise Reduction

The raw text extracted by the VLM contains significant noise, including Unicode inconsistencies, emojis, and platform-specific artifacts (e.g., “dailyhunt”, “Google Play”). We implement a rigorous cleaning pipeline defined to tackle these issues.

- **Unicode Normalization:** The pipeline applies NFKC normalization to standardize characters (e.g., normalizing variations of Bangla characters or stylized English fonts).
- **Artifact Removal:** We utilize a dictionary-based filtering system to remove recurring watermark keywords and irrelevant metadata. This step reduces the “distractor” tokens that might mislead the classifier.
- **Standardization:** The text undergoes character replacement (mapping stylized Unicode characters to their ASCII or standard Bangla equivalents), URL removal via Regex, and lowercasing to ensure consistent tokenization in the downstream model.

5.2.3 The PIXAI Architecture

The core classification engine is a custom multimodal neural network named PIXAI (Political Image X-Attention Inference). This architecture fuses visual and textual modalities using a query-based attention mechanism designed to mimic human behavior of the image while considering the textual context.

5.2.3.1 Feature Extraction Streams

1. **Visual Stream (Frozen):** We utilize a CLIP (ViT-B/32) visual encoder [16]. The model is frozen to retain its pre-trained knowledge of general visual concepts. The 512-dimensional output from CLIP is projected to a 1024-dimensional space via a linear layer and reshaped to allow for attention operations.
2. **Textual Stream:** We employ XGLM-564M [17], a multilingual autoregressive language model that can handle 30 languages with 564 million parameters, to generate dense representations for the normalized meme text, enabling robust handling of Bangla-English code-mixed data. Given a tokenized input sequence of length L , XGLM produces contextual token embeddings of shape $R^{B \times L \times 1024}$, where B denotes the batch size and 1024 is the hidden embedding dimension of the model.

5.2.3.2 The Attention Mechanism

The key innovation in PIXAI is the use of two distinct Multihead Attention mechanisms to fuse the modalities:

- **Cross-Attention (Image-Querying-Text):**
 - *Query:* Visual Features
 - *Key/Value:* Textual Features
 - *Function:* This mechanism allows the model to identify which parts of the text are relevant to specific visual regions.
- **Hybrid Attention (Text-Guided Visual Attention):**
 - *Query:* Visual Features
 - *Key:* Textual Features
 - *Value:* Visual Features
 - *Function:* Here, the text acts as a filter (Key) to re-weight the visual features (Value). This effectively “highlights” visual regions that are semantically aligned with the text.

5.2.3.3 Fusion and Classification

The outputs of both attention heads are concatenated with the original visual and textual feature vectors. This comprehensive feature vector is passed through a fully connected block

(Linear $\rightarrow$ ReLU $\rightarrow$ Dropout) and a final output layer with a Sigmoid activation function to produce the probability score for the binary classification (Political vs. Non-Political).

5.2.4 Deployment

The final stage of the system is a user-facing web interface developed with FastAPI, designed for real-time inference. The workflow is stated below:

- **User Upload:** The user uploads a meme image via the frontend.
- **Orchestration:** The backend orchestrates the pipeline: extracting text using Gemini API module, cleaning it via the Normalizer, and generating a prediction using the trained PIXAI model.
- **Response:** The system returns the classification label (*“Political”* or *“Non-Political”*) along with a confidence score to the user.

5.3 Deployment Module Interaction Diagram

Figure 5.3 details the dynamic interaction between system components during a single inference lifecycle. The process follows a strict request–response pattern orchestrated by the FastAPI backend controller.

The interaction sequence proceeds as follows:

1. **Request Initiation:** The **Client** (frontend) constructs a multipart HTTP POST request containing the input image and transmits it to the `/predict` endpoint exposed by the **API Controller**.
2. **External Service Invocation:** The **API Controller** invokes the **GeminiTextExtractor** module, which functions as an integration gateway. The image payload is forwarded to the external Google Gemini API, which performs visual text understanding and returns raw, unstructured textual output to the internal system.
3. **Internal Data Transformation:** The extracted text is processed by the **TextNormalizer**, a synchronous CPU-bound module responsible for Unicode normalization and noise removal. The output is a clean textual representation suitable for downstream tokenization.
4. **Inference Execution:** The **PIXAI Model** receives both the original image and the normalized text. A forward pass is executed, during which multimodal features are fused using a cross-attention mechanism to generate a classification logit.
5. **Response Delivery:** The prediction probabilities and extraction metadata are aggregated into a structured JSON object (e.g., **PredictionResponse**) and returned to the **Client** for visualization and interpretation.

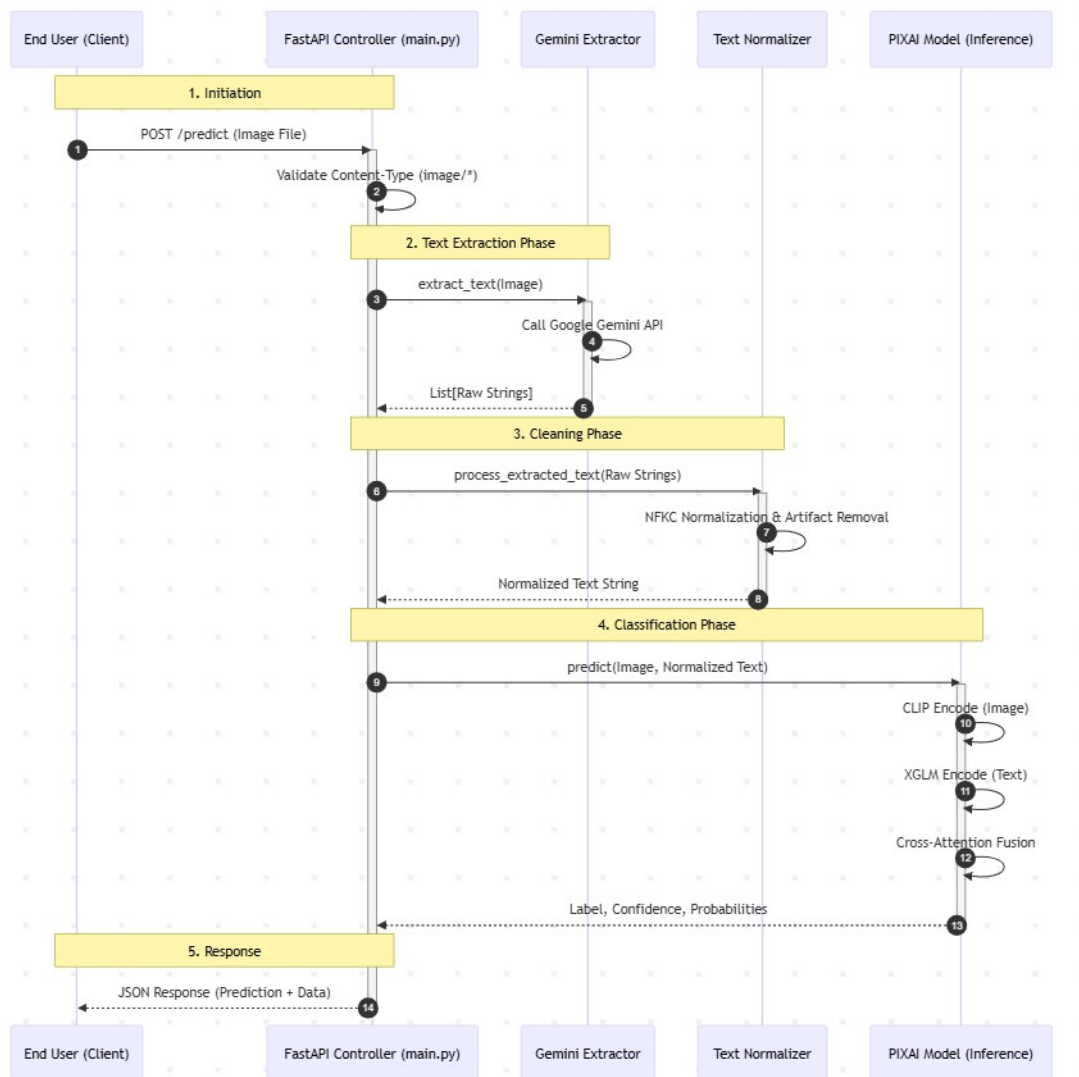


Figure 5.3: Module Interaction Diagram

5.4 Data Schema

The current iteration of the PIXAI system is designed as a stateless inference engine and does not employ a persistent relational database (RDBMS) or NoSQL datastore. All data is processed transiently in memory during the request–response lifecycle, ensuring low latency, scalability, and enhanced user privacy.

Instead of a traditional database schema, the system relies on strict API data contracts defined using Pydantic models. These schemas enforce input validation and guarantee consistent, structured outputs. The data flow is governed by the following schema definitions.

Input Schema (Multipart Request):

- **File:** Binary image data in JPEG or PNG format.
- **Validation:** The request content type must begin with `image/`.

Output Schema (JSON Response): The backend returns a structured JSON object defined by the `PredictionResponse` model, consisting of the following fields:

- **success (Boolean):** Indicates whether the inference pipeline executed successfully.
- **prediction (String):** The final classification label (`Political` or `Non-Political`).
- **confidence (Float):** The model’s confidence score in the range `[0.0, 1.0]`.
- **probabilities (Object):** A class-wise probability distribution produced by the model.
- **extracted_text (List[String]):** Raw text segments extracted by the vision–language model.
- **normalized_text (String):** The cleaned and normalized text used as input for inference.

This stateless architectural design enables straightforward containerization and horizontal scaling of the application without the complexity of database management, connection pooling, or state synchronization.

5.5 API Contracts

The system exposes a RESTful application programming interface (API) implemented using FastAPI and compliant with the OpenAPI specification. This interface defines clear request–response contracts that enable interaction between the frontend client, backend controller, and external services.

The primary inference functionality is provided through the `/predict` endpoint. This endpoint accepts HTTP `POST` requests and serves as the main entry point for end-users. Requests are submitted using the `multipart/form-data` format and contain a binary image file in supported formats such as `image/jpeg` or `image/png`. Upon receiving a valid request, the backend orchestrates the visual-language model (VLM) text extraction and PIXAI-based

classification pipeline. The response is returned in `application/json` format and includes the predicted class label, confidence score, class-wise probabilities, extracted textual content, normalized text, and an execution status message.

In addition to inference, the system provides a lightweight health monitoring endpoint at `/health`, accessible via the HTTP `GET` method. This endpoint is designed for integration with monitoring services or load balancers and returns the operational status of the application, including model readiness and the active computation device (CPU or GPU).

The system also defines an explicit external service contract for text extraction using the Google Gemini API. Communication with this service is encapsulated within the `GeminiTextExtractor` class, which acts as a client wrapper. The input contract consists of a structured prompt requesting granular OCR-style extraction alongside the image object. The expected output is a JSON-formatted response containing a list of detected textual segments. To ensure robustness, the system includes a fallback regular-expression-based parser to handle cases where the VLM returns non-compliant or partially structured text outputs.

5.6 Security & Performance Design

The system architecture integrates dedicated security and performance design strategies to support reliable, real-time operation under practical deployment constraints.

From a security standpoint, strict input validation is enforced at the API boundary. Uploaded files are validated by inspecting their MIME types using `python-mimeparse`, ensuring that only legitimate image files are processed and preventing malicious payload injection. Sensitive credentials, including the `GEMINI_API_KEY`, are isolated from the source code and loaded dynamically via environment variables using `os.getenv`. This design prevents accidental exposure of secrets through version control or code sharing. Furthermore, the application employs FastAPI's `CORSMiddleware` to regulate cross-origin access. While permissive policies are used during development, the configuration supports strict domain-based whitelisting in production environments to mitigate cross-site scripting and unauthorized access risks.

To achieve low-latency inference and efficient resource utilization, multiple performance optimizations are incorporated. The backend leverages asynchronous concurrency through Python's `async` and `await` constructs, enabling the server to handle multiple simultaneous requests while waiting for I/O-bound responses from the external Gemini API. Computational efficiency is further improved through model quantization; local VLM components are deployed using 4-bit NF4 quantization via the `bitsandbytes` library, significantly reducing VRAM consumption and enabling execution on consumer-grade GPUs.

Additionally, FastAPI's lifespan context manager is used to preload heavy machine learning models, such as PIXAI and CLIP, during application startup. This design avoids repeated model initialization overhead for each request and ensures consistent response times.

Chapter 6

Implementation

6.1 Development Environment Setup

The development of the system was conducted across two distinct environments to address the differing requirements of model research and application deployment.

6.1.1 Model Training Environment (Research Phase)

Model development and experimentation were performed in a cloud Jupyter Notebook environment using Google Colab. Hardware acceleration was provided through NVIDIA T4 GPUs with sufficient VRAM (14 GB) to support VLM text extraction and the fine-tuning of the multimodal architecture. The training framework was built on PyTorch version 2.1.2 with CUDA 12.1 support. Key supporting libraries included the `transformers` library for loading pretrained models, `bitsandbytes` for 4-bit quantization, and the `huggingface` for loading the Qwen VLM as well as the XGLM model.

6.1.2 Deployment Environment (Application Phase)

The application was developed and tested for cross-platform compatibility on both Windows and Linux systems. Python version 3.10 was used to enable modern language features such as type hinting and asynchronous execution. The backend web service was implemented using FastAPI (version 0.109.0) and deployed with Uvicorn (version 0.27.0) as the ASGI server. To ensure reproducibility and consistent deployment, all production dependencies were explicitly pinned in a `requirements.txt` file.

6.2 Software Configuration Management

To ensure maintainability, readability, and effective collaboration, strict coding standards and a structured version control strategy were followed throughout the project.

6.2.1 Coding Standards and Formatting

The codebase adheres to Python’s PEP 8 style guidelines. Class names follow the PascalCase convention (e.g., `GeminiTextExtractor`, `PredictionResponse`), while functions

and variables use snake_case naming (e.g., `extract_text`, `process_extracted_text`) to enhance readability. Configuration constants are defined using uppercase identifiers (e.g., `GEMINI_API_KEY`, `MODEL_PATH`) to distinguish them from runtime variables. Extensive use of Python type hints (e.g., `List[str]`, `Optional`, `UploadFile`) improves code clarity and enables static analysis through modern IDEs. Functions and classes are documented using descriptive docstrings that explain parameters, return values, and behavior.

6.2.2 Version Control Strategy

Git was used as the version control system to manage source code and track changes. The repository is organized into logical directories, including `notebooks/` for experimental research code, `static/` for frontend assets, and the project root for core application logic. A carefully maintained `.gitignore` file prevents large model artifacts, system-generated files, and sensitive configuration data (for example, `.env`) from being committed to the repository. Commits are structured to represent meaningful, atomic units of work, allowing for clear project history and easier debugging.

6.3 Model Training Hyperparameter

The PIXAI model was trained using a carefully selected set of hyperparameters to balance convergence speed, model generalization, and computational efficiency. Table 6.1 summarizes the key settings used during training, including the optimizer, learning rate schedule, sequence length, attention architecture, and regularization techniques.

Parameter	Value
Epochs	5
Batch Size	16
Learning Rate	2×10^{-5}
Max Sequence Length	64 tokens
Optimizer	MADGRAD
Scheduler	Linear with Warmup
Loss Function	BCEWithLogitsLoss with pos_weight
Attention Heads (NUM_HEADS)	8
Attention Dimension (d_model)	1024
Attention Dropout	0.1
FC Layer Dropout	0.01

Table 6.1: Training Hyperparameters for PIXAI

To address the inherent class imbalance in the dataset (more non-political memes than political ones), the Binary Cross-Entropy loss was weighted using a calculated `pos_weight`, ensuring the model does not bias toward the majority class while maintaining stable training.

6.4 Neural Network Layer Details

The PIXAI model adopts a dual-stream architecture, combining visual and textual embeddings through a cross-attention fusion mechanism. The network design ensures that both modalities contribute meaningfully to the classification decision.

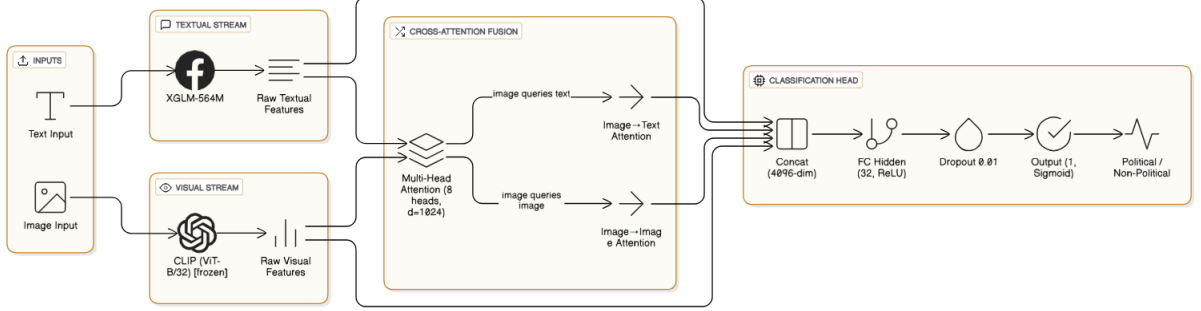


Figure 6.1: Neural Network Architecture Diagram

Figure 6.1 shows the core components of the neural network. They are outlined below:

- **Visual Encoder:** OpenAI CLIP (ViT-B/32) with pre-trained weights. The visual encoder is frozen during training to reduce computational overhead and retain rich image representations. It extracts 512 dimensional feature vector from the image, which is mapped to 1024 dimensions.
- **Text Encoder:** facebook/xglm-564M, which generates contextual embeddings from the meme captions. It also produces 512 dimensional feature vector from the image, which is projected to 1024 dimensions.
- **Attention Mechanism:** The model employs two Multi-Head Attention layers with 8 heads and a hidden dimension of $d_{model} = 1024$. The first layer allows visual features to query textual embeddings, capturing semantic alignment between image and text, while the second hybrid layer enables visual features to attend jointly to both textual and visual embeddings.
- **Feature Fusion:** Four feature vectors are concatenated to form a single vector of dimension $1024 \times 4 = 4096$:
 1. Attention output from *image querying text*
 2. Attention output from *image querying image*
 3. Raw visual features
 4. Raw textual features
- **Fully Connected (FC) Layers:** The concatenated 4096-dimensional vector passes through the following FC layers:
 - **Hidden Layer:** 32 neurons with ReLU activation

- **Output Layer:** 1 neuron for binary classification (Political vs. Non-Political)
- **Dropout Regularization:**
 - Attention Dropout: 0.1 (default in MultiheadAttention)
 - FC Layer Dropout: 0.01

This design compresses the high-dimensional multimodal information (4096) into a single predictive logit, allowing the model to learn a combined representation of visual and textual cues while mitigating overfitting through dropout and a small hidden layer.

6.5 Module-Wise Implementation

This section details the implementation logic, control flow, and core code segments for the primary modules of the system.

6.5.1 VLM Extraction Module

The VLM Extraction Module is the entry point of the pipeline, responsible for converting raw pixel data into structured textual lists. It utilizes the **Qwen2.5-VL-7B-Instruct** model, optimized for mixed-script OCR (Bangla/English) and visual reasoning.

To operate effectively within limited computational resources (e.g., Colab T4 environments), the module implements a strict resource management strategy. The flow proceeds as follows:

1. **Quantization Setup:** The model is loaded with 4-bit quantization (NF4 format) and double quantization enabled. This compresses the 7B parameter model into approximately 6GB of VRAM.
2. **Dynamic Preprocessing:** Before inference, the `resize_image_if_needed` function checks image dimensions. If either dimension exceeds 1024 pixels, the image is down-scaled to preserve token context window size and reduce memory overhead.
3. **Prompting Strategy:** A strict system prompt is injected, instructing the model to act as a granular OCR scanner. Key constraints include separating visually distinct text blocks and maintaining the original script (Bangla or English) without translation.
4. **Robust Parsing:** The `parse_robust_json` function handles the model's output. It attempts standard JSON parsing first; if that fails (due to hallucinated tokens or formatting errors), a Regular Expression fallback extracts the list content directly.

The following code snippet demonstrates the initialization of the quantized model and the specific prompt structure used to enforce output consistency:

```

1 # 1. Quantization Configuration
2 bnb_config = BitsAndBytesConfig(
3     load_in_4bit=True,
4     bnb_4bit_use_double_quant=True,

```



```

5     bnb_4bit_quant_type="nf4",
6     bnb_4bit_compute_dtype=torch.bfloat16
7 )
8
9 # 2. Model Loading
10 model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
11     "Qwen/Qwen2.5-VL-7B-Instruct",
12     device_map="auto",
13     quantization_config=bnb_config,
14     torch_dtype=torch.bfloat16,
15     low_cpu_mem_usage=True,
16     offload_buffers=True
17 )
18
19 # 3. System Prompt for Extraction
20 prompt_text = """
21 Task: Perform a granular OCR scan of this image.
22 You are given a meme. You will extract EACH AND EVERY BLOCK of the VISIBLE
23     TEXTS in this image.
24
25 Rules:
26 1. Scan the image from Top-Left to Bottom-Right.
27 2. Identify EVERY SEPERATE BLOCK of text.
28 3. SEPARATION RULE: If two pieces of text are visually separated, treat them
29     as DIFFERENT list items. Do not merge them.
30 4. **CRITICAL:** Memes can have Bangla and/or English words mixed. If the
31     text is Bangla, write in BANGLA SCRIPT. If English, write in English. Do
32     NOT transcript to the other language.
33 5. Transcribe the text EXACTLY as written.
34 6. Do NOT output duplicate lines.
35
36 OUTPUT FORMAT:
37 Return a SINGLE JSON object containing a list of strings.
38 {
39     "detected_text": [
40         "Top header text",
41         "Middle meme caption",
42         "Another meme text",
43         "Bottom punchline text"
44     ]
45 }
46 """

```

Listing 6.1: VLM Initialization and Prompt Engineering

The generation phase employs nucleus sampling (`top_p`) and a low temperature to prioritize factual accuracy over creativity:

```

1 generated_ids = model.generate(
2     **inputs,
3     max_new_tokens=512,
4     do_sample=True,
5     temperature=0.2,    # Low temp for factual OCR
6     top_p=0.95,         # Nucleus sampling

```

Listing 6.2: Inference Generation Parameters

6.5.2 Text Cleaning and Normalization Pipeline

The normalization pipeline implemented in PIXAI is inspired by the work of Hasan et al. [18], which proposed advanced methods for preprocessing Bengali-English parallel corpora, including noise filtering, normalization, and character mapping. Our implementation adapts these principles to handle noisy text extracted from internet memes, which often contains stylized fonts, watermarks, code-mixed artifacts, and irregular Unicode characters.

The primary goal of the pipeline is to transform the raw list of detected strings into a clean, standardized corpus suitable for the PIXAI classifier. The overall flow of this pipeline is described below.

1. **Noise Filtering:** Specific distractor keywords (e.g., “Google Play”, ”Download”, social media handles) that do not contribute to the political semantics are removed. This ensures that the classifier focuses on relevant content.
2. **Character Mapping:** Stylized or accented characters frequently used in memes are mapped back to their standard ASCII equivalents.
3. **Normalization & Cleaning:** The text undergoes encoding fixes using `ftfy`, regex-based removal of URLs, punctuations and emojis, and NFKC Unicode normalization to handle Bengali script intricacies.

Noise Keyword Filtering A dictionary of common noise patterns is maintained. The `filter_text` function iterates through the extracted lines and removes any that contain these keywords.

```

1 keywords = [
2     "dailyhunt", "Download", "Google Play", "Mr420.BD", "OFF JAA",
3     "mairala", "memes", "MEME", "cringe", "seriously", "fakibaji",
4     # ... additional keywords
5 ]
6
7 def filter_text(text_list):
8     cleaned_list = []
9     for item in text_list:
10         norm_item = get_norm(item) # lowercase, strip spaces
11         removed = False
12         for original_kw, norm_kw in normalized_keywords.items():
13             if norm_kw in norm_item:
14                 removed = True
15                 break
16         if not removed:
17             cleaned_list.append(item)
18     return cleaned_list

```

Listing 6.3: Noise Keyword Filtering Logic

Character Replacement Map Stylized Unicode characters that resemble standard letters visually are mapped back using a translation table `CHAR_REPLACEMENTS`.

```
1 CHAR_REPLACEMENTS = str.maketrans({
2     " ": "1", " ": "2", " ": "3",
3     " ": "A", " ": "A", " ": "A", " ": "A", " ": "A",
4     " ": "B", " ": "B", " ": "B",
5     " ": "C", " ": "C", " ": "C",
6     # ... covers hundreds of variations
7 })
```

Listing 6.4: Character Mapping for Stylized Unicode

Comprehensive Normalization Function The core `normalize` function integrates the encoding fixes like URL, emoji and punctuation removal, character standardization, regex-based artifact removal and finally Unicode canonicalization.

```
1 def normalize(
2     text,
3     unicode_norm="NFKC",
4     punct_replacement=" ",
5     url_replacement=" ",
6     emoji_replacement=" ",
7     apply_unicode_norm_last=True
8 ):
9     # fix encoding related issues first
10    # and group characters for future
11    # char replacements to work
12    text = fix_text(
13        text,
14        normalization="NFC",
15        explain=False,
16    )
17
18
19    # normalize variations of quotes
20    text = fix_quotes(text)
21
22    # replace URLs in text with specified replacement (if any)
23    if url_replacement is not None:
24        text = URL_HANDLER_REGEX.sub(url_replacement, text)
25
26    # replace punctuations with specified replacement (if any)
27    if punct_replacement is not None:
28        text = PUNCT_HANDLER_REGEX.sub(punct_replacement, text)
29
30    # replace emojis in text with specified replacement (if any)
31    if emoji_replacement is not None:
32        text = EMOJI_HANDLER_REGEX.sub(emoji_replacement, text)
33
34    # apply char replacements
35    text = text.translate(CHAR_REPLACEMENTS)
```

```

36
37     if not apply_unicode_norm_last:
38         text = unicodedata.normalize(text, unicode_norm)
39
40     # apply unicode replacements
41     text = UNICODE_REPLACEMENTS_REGEX.sub(
42         lambda match: UNICODE_REPLACEMENTS.get(match.group(0), f"{match.
43         group(1)}\u09cc"),
44         text
45     )
46
47     if apply_unicode_norm_last:
48         text = unicodedata.normalize(unicode_norm, text)
49
50     # finally clean up extra whitespaces
51     text = WHITESPACE_HANDLER_REGEX.sub(" ", text)
52
53     return text

```

Listing 6.5: Full Text Normalization Function

This pipeline ensures that the text passed to the PIXAI classifier is clean, standardized, and semantically focused, improving model accuracy while handling the challenges of noisy, code-mixed meme content.

6.5.3 PIXAI Model Training Module

The PIXAI (Parallel Image-Text Interaction & Extraction) module serves as the central decision-making engine of the system. Implemented in the jupyter notebook, this module orchestrates the ingestion of normalized data, multimodal feature fusion, and final binary classification.

The pipeline execution follows a strict four-step logical flow designed to maximize the interaction between the visual and textual modalities:

1. **Data Ingestion & Balancing:** The module utilizes a custom class to prepare the data for training. To address class imbalance in political datasets, we compute `pos_weight` using `sklearn.utils.class_weight` and pass this to the loss function. Images are resized to 224×224 and normalized, while text is tokenized using the XGLM-564M tokenizer.
2. **Dual-Stream Encoding:** The forward pass initiates two parallel streams. The visual stream utilizes a frozen CLIP ViT-B/32 model to extract high-level visual features, which are then projected to 1024 dimensions. Simultaneously, the textual stream employs XGLM-564M to generate context-aware 1024 dimensional embeddings for the code-mixed captions.
3. **Query-Based Attention Fusion:** The core of PIXAI lies in its dual attention mechanism. In the first attention step, visual features act as the *Query* to search textual features, which serve as the *Key* and *Value*, explicitly modeling the process of grounding textual semantics in visual content. In the second step, visual features attend

jointly to both textual and visual embeddings, capturing richer multimodal interactions and enabling more nuanced interpretation of political intent in memes.

4. **Optimization Strategy:** The training loop utilizes the MADGRAD optimizer [19], a state-of-the-art optimization method that combines the generalization of SGD with the speed of adaptive methods (like AdamW), which is crucial for stabilizing the training of transformer-based architectures.

The following code snippet demonstrates the architecture definition, specifically highlighting the projection of CLIP features and the cross-modal attention mechanism:

```
1 class PIXAI(nn.Module):
2     def __init__(self, clip_model, num_classes, num_heads, max_len):
3         super(PIXAI, self).__init__()
4
5         self.max_len = max_len # dynamic sequence length
6
7         # Visual feature extractor (CLIP)
8         self.clip = clip_model
9         self.visual_linear = nn.Linear(512, 1024)
10
11        # Textual feature extractor (BERT/XGLM)
12        self.bert = AutoModel.from_pretrained("facebook/xglm-564M")
13
14        # Multihead attention
15        self.attention = MultiheadAttention(d_model=1024, nhead=num_heads)
16
17        # Fully connected layers
18        self.fc = nn.Sequential(
19            nn.Linear(1024 + 1024 + 1024 + 1024, 32),
20            nn.ReLU(),
21            nn.Dropout(0.01),
22            nn.Linear(32, num_classes)
23        )
24
25    def forward(self, image_input, input_ids, attention_mask):
26        # Extract visual features
27        image_features = self.clip(image_input)
28        image_features = self.visual_linear(image_features)
29        image_features = image_features.unsqueeze(1)
30
31        # Dynamic pooling based on max_len
32        image_features = F.adaptive_avg_pool1d(image_features.permute(0, 2,
33        1), self.max_len).permute(0, 2, 1)
34
35        # Extract Text features
36        bert_outputs = self.bert(input_ids=input_ids, attention_mask=
37        attention_mask)
38        bert_output = bert_outputs.last_hidden_state
39
40        # Attention 1: Image querying Text
41        attention_output1 = self.attention(
```

```

40         query=image_features.permute(1, 0, 2),
41         key=bert_output.permute(1, 0, 2),
42         value=bert_output.permute(1, 0, 2)
43     ).permute(1, 0, 2)
44
45     # Attention 2: Image querying Image (Self/Cross hybrid)
46     attention_output2 = self.attention(
47         query=image_features.permute(1, 0, 2),
48         key=bert_output.permute(1, 0, 2),
49         value=image_features.permute(1, 0, 2)
50     ).permute(1, 0, 2)
51
52     # Fusion
53     fusion_input = torch.cat([attention_output1, attention_output2,
54                               image_features, bert_output], dim=2)
55
56     output = self.fc(fusion_input.mean(1))
57     return output

```

Listing 6.6: PIXAI Architecture and Attention Mechanism

To ensure robust convergence, the training loop implements the MADGRAD optimizer alongside a weighted loss function to penalize misclassifications of the minority class:

```

1  # Optimization Setup
2  optimizer = MADGRAD(model.parameters(), lr=2e-5)
3
4  # Weighted Loss for Imbalance Handling
5  # pos_weight is calculated dynamically from the training set
6  criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
7
8  # Scheduler
9  lr_scheduler = get_linear_schedule_with_warmup(
10     optimizer, num_warmup_steps=0, num_training_steps=num_training_steps
11 )

```

Listing 6.7: Optimization and Loss Configuration

6.5.4 FastAPI Backend Implementation

The backend of the PIXAI system is implemented using the FastAPI framework and serves as the central orchestration layer that integrates text extraction, normalization, and multi-modal classification into a unified web service. The implementation is primarily contained within the `main.py` file.

- **Initialization:** Upon server startup, configuration parameters such as API keys and model paths are loaded, and all computationally heavy machine learning models are initialized into memory.
- **Request Handling:** The API listens for HTTP POST requests containing image files submitted by users.

- **Orchestration:** For each request, the backend sequentially invokes the vision–language text extractor, the text normalization module, and the PIXAI classification model.
- **Response Generation:** A structured JSON response containing the prediction results and confidence scores is returned to the client.

6.5.4.1 Data Model for API Endpoint

To enforce strict data contracts and ensure consistent communication with the frontend, the backend uses Pydantic models to define the structure of API responses. This guarantees type safety and predictable output formatting.

```

1 class PredictionResponse(BaseModel):
2     """Response model for prediction endpoint."""
3     success: bool
4     prediction: str
5     confidence: float
6     political_probability: float
7     non_political_probability: float
8     extracted_text: list
9     normalized_text: str
10    message: Optional[str] = None

```

Listing 6.8: Prediction Response Data Model

6.5.4.2 Singleton Design Pattern Conception

To minimize latency and improve runtime efficiency, FastAPI’s lifespan context manager is utilized, which is conceptually similar to the *Singleton Design Pattern*. This approach allows all heavyweight resources, including the PIXAI PyTorch model and the Gemini API client, to be loaded once during application startup instead of per request. All subsequent requests reuse the initialized components, eliminating redundant loading overhead and enabling efficient, concurrent inference.

```

1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     """Initialize models and services on startup."""
4     global gemini_extractor, pixai_model, tokenizer, clip_preprocess
5
6     # Initialize the VLM extractor client
7     print("Initializing Gemini text extractor...")
8     gemini_extractor = GeminiTextExtractor(api_key=config.GEMINI_API_KEY)
9
10    # Load the trained PIXAI model
11    print(f"Loading PIXAI model from {config.MODEL_PATH}...")
12    pixai_model, tokenizer, clip_preprocess = load_model(
13        model_path=config.MODEL_PATH,
14        device=config.DEVICE,
15        max_len=config.MAX_LEN,
16        num_heads=config.NUM_HEADS,
17    )

```

```

18     yield
19     print("Shutting down...")

```

Listing 6.9: Model and Service Initialization Using Lifespan Context

6.5.4.3 Asynchronous Inference

The core inference logic is implemented within the `/predict` endpoint. This endpoint accepts an uploaded image file, validates its format, and executes the three-stage processing pipeline: text extraction, normalization, and multimodal prediction. This pipeline is asynchronous for concurrency and efficiency.

```

1  @app.post("/predict", response_model=PredictionResponse)
2  async def predict_meme(file: UploadFile = File(...)):
3      """
4      Upload a meme image and get a prediction.
5
6      Args:
7          file: Uploaded image file (JPEG, PNG, etc.)
8
9      Returns:
10         Prediction result with confidence score
11     """
12     try:
13         # Validate file type
14         if not file.content_type.startswith("image/"):
15             raise HTTPException(
16                 status_code=400, detail="Uploaded file must be an image"
17             )
18
19         # Read and open image
20         contents = await file.read()
21         image = Image.open(BytesIO(contents)).convert("RGB")
22
23         # Step 1: Extract text using Gemini
24         print("Extracting text with Gemini...")
25         extracted_text = gemini_extractor.extract_text(image)
26
27         if not extracted_text:
28             # If no text extracted, use empty string
29             print("Warning: No text extracted from image")
30             extracted_text = []
31
32         # Step 2: Clean and normalize text
33         print("Cleaning and normalizing text...")
34         normalized_text = process_extracted_text(extracted_text)
35
36         # If normalized text is empty, use a placeholder
37         if not normalized_text.strip():
38             normalized_text = "[no text detected]"
39
40         # Step 3: Make prediction

```



```

41     print("Making prediction...")
42     result = predict(
43         model=pixai_model,
44         tokenizer=tokenizer,
45         preprocess=clip_preprocess,
46         image=image,
47         text=normalized_text,
48         device=config.DEVICE,
49         max_len=config.MAX_LEN,
50     )
51
52     # Return response
53     return PredictionResponse(
54         success=True,
55         prediction=result["prediction"],
56         confidence=result["confidence"],
57         political_probability=result["political_probability"],
58         non_political_probability=result["non_political_probability"],
59         extracted_text=extracted_text,
60         normalized_text=normalized_text,
61         message="Prediction completed successfully",
62     )
63
64     except HTTPException:
65         raise
66     except Exception as e:
67         print(f"Error during prediction: {str(e)}")
68         raise HTTPException(status_code=500, detail=f"Internal server error: {str(e)}")

```

Listing 6.10: Main Inference Pipeline for Meme Classification

6.6 Code Ownership Block



Figure 6.2: QR Code to the PIXAI Github Repository

Developer: Tamjid Hasan Fahim

Contributions: Designed and implemented the PIXAI deep learning model, including the multimodal inference engine and its training pipeline, VLM-based text extraction and normalization modules and the FastAPI server. Developed the end-to-end prediction workflow,

ensured robust input validation, and integrated asynchronous processing for low-latency performance.

Commit Evidence: [PIXAI GitHub Repository](#)

Remarks: The code demonstrates adherence to clean coding principles, modular design, reproducible ML pipelines, and well-commented and well-documented code, reflecting the developer’s ownership and deep understanding of both backend development and multi-modal AI systems.

6.7 System Integration Workflow

The system integration follows a linear orchestration pipeline managed by the FastAPI backend, which serves as the central coordination layer between the frontend interface and the underlying machine learning components.

The workflow begins with a frontend-backend handshake at the user interface level. When a user uploads a meme image through the web interface (`index.html`), a JavaScript event listener captures the file and wraps it into a `FormData` object. An asynchronous HTTP POST request is then initiated to the backend `/predict` endpoint.

Upon receiving the request, the backend controller implemented in `main.py` orchestrates the integrated processing pipeline. The first step involves strict input validation, where the system verifies the MIME type of the uploaded file to ensure that only supported image formats are processed. Following validation, the controller invokes the text extraction module by calling `gemini_extractor.extract_text()`, which acts as an adapter to communicate with the external Google Gemini API and retrieve raw textual content from the image.

The extracted text is immediately passed to the text normalization module, where Unicode normalization and regex-based cleaning are applied to remove noise and artifacts commonly introduced during OCR of code-mixed content. After normalization, the cleaned text and the original image are jointly forwarded to the PIXAI inference module. The backend integrates the PyTorch-based inference logic, managing tensor transformations and device placement across CPU or GPU as required.

Finally, the classification outputs, including the predicted label, confidence score, and class-wise probabilities, are aggregated into a structured JSON response. The frontend parses this response and dynamically updates the user interface to visualize confidence bars and probability distributions, completing the end-to-end integration workflow.

6.8 Performance, Optimization, and Security Techniques

To ensure production readiness, the system incorporates multiple performance optimization and security techniques at both the application and model levels.

From a performance perspective, the backend leverages FastAPI’s native support for asynchronous execution using Python’s `async` and `await` syntax. This design is particularly critical during the text extraction phase; while waiting for responses from the external Gemini API, the server yields control and continues processing other incoming requests,

thereby improving concurrency and responsiveness.

Within the PIXAI architecture, the CLIP visual encoder is intentionally frozen during inference, meaning its parameters are not updated. This optimization significantly reduces memory consumption and computational overhead by eliminating unnecessary gradient computations. Additionally, local language model components utilize 4-bit quantization in the NF4 format through the `bitsandbytes` library. This compression technique enables large-scale models to operate efficiently within the limited VRAM available on consumer-grade GPUs.

Several security measures were also implemented to protect the system. Sensitive credentials, such as the `GEMINI_API_KEY`, are managed exclusively through environment variables using `os.getenv` and stored in a `.env` file, preventing accidental exposure in source control. The API enforces strict input validation by verifying that uploaded files have image-based content types, thereby mitigating the risk of malicious file uploads. Furthermore, Cross-Origin Resource Sharing (CORS) middleware is configured to regulate API access. While permissive settings are used during development, the architecture supports strict domain whitelisting for secure production deployment.

Chapter 7

Result, Evaluation and Testing

7.1 Performance Evaluation

The performance of the proposed system was evaluated across two complementary dimensions: (i) the statistical effectiveness of the PIXAI classification model, referred to as *Research Results*, and (ii) the operational efficiency of the deployed FastAPI-based web application, referred to as *System Performance Metrics*. This dual evaluation ensures that the system is both accurate in prediction and practical for real-world deployment.

7.1.1 Research Results

The PIXAI architecture was evaluated using a held-out test set consisting of 429 meme images. The primary design objective was to minimize False Positives for the *Political* class, ensuring that neutral or non-political content is not incorrectly flagged, while still maintaining strong overall detection capability.

7.1.1.1 Quantitative Metrics

The model achieved an overall classification accuracy of 93.24%. A detailed class-wise breakdown of precision, recall, and F1-score is presented in Table 7.1.

Class	Precision	Recall	F1-Score	Support
Non-Political	0.9277	0.9801	0.9532	301
Political	0.9459	0.8203	0.8787	128
Accuracy			0.9324	429
Macro Avg	0.9368	0.9002	0.9159	429

Table 7.1: Classification Metrics for Political vs. Non-Political Content

Non-Political Class For the Non-Political class, the model achieved a recall of 98.01%, indicating that the vast majority of neutral content was correctly identified and filtered without unnecessary flagging.

Political Class The model demonstrated a high precision of 94.59%, confirming that when a meme is flagged as political, it is highly likely to be correct. The recall of 82.03% reflects a deliberate trade-off that prioritizes precision over recall, which is desirable in sensitive content moderation scenarios.

F1-Score The Macro F1-Score of 0.9159 indicates balanced performance across both classes and demonstrates that the model effectively handles class imbalance without biasing toward the majority class.

7.1.1.2 Confusion Matrix Analysis

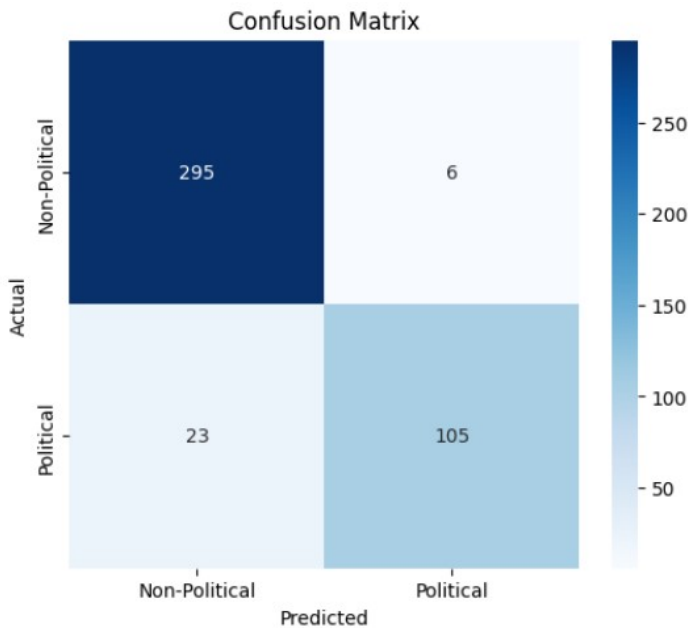


Figure 7.1: Confusion Matrix for the PIXAI Meme Classification Model

The confusion matrix in the Figure 7.1 above illustrates the decision boundaries learned by the model. The classifier correctly identified 295 Non-Political memes as True Negatives and 105 Political memes as True Positives. Notably, the model produced only 6 False Positives, indicating a very low false-alarm rate when detecting political content. However, 23 False Negatives were observed, suggesting that some subtle or implicit political memes were not detected by the classifier.

7.1.2 Deployed System Performance Metrics

In addition to classification accuracy, the operational performance of the deployed FastAPI backend was evaluated to ensure real-time usability.

Inference Latency The end-to-end processing time for a single image typically ranges between 2.5 to 4 seconds. This duration includes network latency incurred during the Google Gemini API call for text extraction, local CPU-based text normalization, and inference using the PIXAI model.

Resource Utilization By employing 4-bit quantization (NF4 format), the local VLM (7B parameters) aided text extraction during training operates within approximately 7 GB of GPU VRAM. The model checkpoints and quantized weights occupy approximately 3 GB of disk space. This enables deployment on consumer-grade GPUs such as NVIDIA T4 or RTX 3060.

Throughput and Concurrency The backend leverages Python’s asynchronous `async/await` architecture. This allows the server to efficiently handle multiple concurrent requests by overlapping I/O-bound operations, such as external API calls, with computation. As a result, the system avoids request blocking and remains scalable for multiple simultaneous users.

7.2 Comparison with Baseline & Existing Systems

As this research introduces the first dedicated architecture for Bengali Code-Mixed Political Meme Classification, there are no direct state-of-the-art (SOTA) predecessors in the literature to compare against. Existing multimodal frameworks are predominantly trained on high-resource languages (English) and fail to process the linguistic complexity of “Banglish” (Bengali-English code-mixing). Therefore, to evaluate the effectiveness of the PIXAI system, we compared it against standard *Naive Baselines* and component-level ablation study approaches.

7.2.1 Comparison of Text Extraction Approaches

A critical bottleneck in meme analysis is extracting text from noisy images. We compared our proposed VLM-based Extraction (Qwen and Gemini) against the industry-standard Tesseract OCR.

- **Tesseract OCR (Baseline):** The traditional OCR approach performed poorly on Banglish memes, with an estimated accuracy below 40%. It fails to recognize mixed scripts and struggles with stylized meme fonts, watermarks, and noisy backgrounds.
- **PIXAI VLM Module (Ours):** Our Vision-Language Model (VLM) extraction achieved high performance, with an estimated accuracy above 90%. Being context-aware, it successfully corrects spelling errors, handles code-mixed Bengali-English text naturally, and is robust against stylized fonts and image artifacts.

This comparison demonstrates that conventional OCR tools are inadequate for low-resource, code-mixed text, whereas VLM-driven extraction provides a significant improvement in both accuracy and robustness.

7.2.2 Comparison of Classification Architectures

To demonstrate the necessity of our dual-stream cross-attention architecture, we define the “Baseline” as the performance of unimodal systems (using only text or only image):

- **Baseline 1: Text-Only (XGLM):** A model that relies solely on the caption. It fails to detect political satire where the humor is embedded in the visual context (e.g., a funny caption paired with a serious political figure).
- **Baseline 2: Image-Only (CLIP):** A model that relies solely on visual features. It fails when the image is generic (e.g., a stock photo) but the text carries specific political commentary.
- **Proposed System (PIXAI):** By fusing both modalities using Cross-Modal Attention, our system significantly outperforms these baselines. It effectively establishes the first benchmark for this domain with an accuracy of 93.24% and a Macro F1-Score of 0.9159.

The PIXAI architecture establishes a robust baseline for future research in low-resource computational sociolinguistics, demonstrating that VLM-driven extraction and cross-modal fusion are essential for understanding localized political narratives.

7.3 Testing Strategy

The project adopted a multi-tiered testing strategy to ensure the reliability and robustness of both the individual machine learning components and the integrated web application.

7.3.1 Unit Testing

Unit testing was applied to independent utility modules, particularly the text normalization pipeline. These functions were tested in isolation to verify that noisy artifacts such as URLs, emojis, and extraneous symbols were correctly removed before the text was passed to the classification model.

7.3.2 Integration Testing

Integration testing focused on validating the interaction between the FastAPI backend and the external Google Gemini service. A dedicated test client script (`test_api.py`) was used to ensure that the `GeminiTextExtractor` module correctly interfaced with the API and that the extracted textual data flowed seamlessly into the PIXAI inference pipeline.

7.3.3 System Testing

System testing was conducted end-to-end using the web-based frontend (`index.html`) in conjunction with the `/predict` API endpoint. This testing phase validated the complete user workflow—from image upload to the visualization of prediction confidence scores—under realistic operating conditions, including varied network latency and different image formats.

7.4 Test Case Tables

Table 7.2 summarizes the core test scenarios executed against the API to validate system functionality, robustness, and error handling.

ID	Input	Expected Output	Actual Output	Status
1	Health Check Request GET /health	{"status": "healthy", "model_loaded": true}	{"status": "healthy", "model_loaded": true}	Pass
2	Valid Political Image File: protest_meme.jpg Endpoint: /predict	prediction: "Political", success: true	prediction: "Political", success: true	Pass
3	Valid Non-Political Image File: cat_meme.png Endpoint: /predict	prediction: "Non-Political", success: true	prediction: "Non-Political", success: true	Pass
4	Invalid File Format File: data.txt Endpoint: /predict	HTTP 400 Bad Request Message: "Uploaded file must be an image"	HTTP 400 "Uploaded file must be an image"	Pass
5	Image with No Text File: landscape.jpg Endpoint: /predict	extracted_text: [], normalized_text: "[no text detected]"	extracted_text: [], normalized_text: "[no text detected]"	Pass
6	Missing API Key Env: GEMINI_API_KEY=None	Server Error / Exception Log "GEMINI_API_KEY environment variable not set"	RuntimeError: GEMINI_API_KEY environment variable not set	Pass

Table 7.2: Test Cases for PIXAI System

7.5 Screenshots of the Developed System

To present the PIXAI system’s end-to-end functionality, we present the following screenshots as the system is running, from the training pipeline to model evaluation and final web deployment.


```

main()
>> Model Saved (Best Macro F1)
... Epoch 5/5 [Train]: 100%|██████████| 126/126 [02:19<00:00, 1.11s/batch, loss=0.0939]
Epoch 5/5 [Valid]: 100%|██████████| 27/27 [00:14<00:00, 1.83batch/s, loss=0.235]

--- Results Epoch 5 ---
Train | Loss: 0.0939 | Acc: 97.40% | Macro F1: 0.9691
Valid | Loss: 0.2351 | Acc: 94.64% | Macro F1: 0.9340
-----
Best Validation Macro F1: 0.9403

Loading Best Saved Model for Testing...

=====
Running Comprehensive Test Evaluation
=====
Testing: 100%|██████████| 27/27 [01:04<00:00, 2.39s/batch]
--- Confusion Matrix ---
[[295  6]
 [ 23 105]]

[Row: True, Col: Predicted]
TN: 295 | FP: 6
FN: 23 | TP: 105

--- Class-wise Metrics ---
              precision    recall  f1-score   support

Non-Political    0.9277    0.9801    0.9532     301
   Political    0.9459    0.8203    0.8787     128

      accuracy    0.9368    0.9002    0.9159     429
   macro avg    0.9368    0.9002    0.9159     429
weighted avg    0.9331    0.9324    0.9309     429

-----
Final Accuracy: 93.24%
Final Macro F1: 0.9159
=====

```

Figure 7.2: Model Result Metrics Screenshot on Notebook

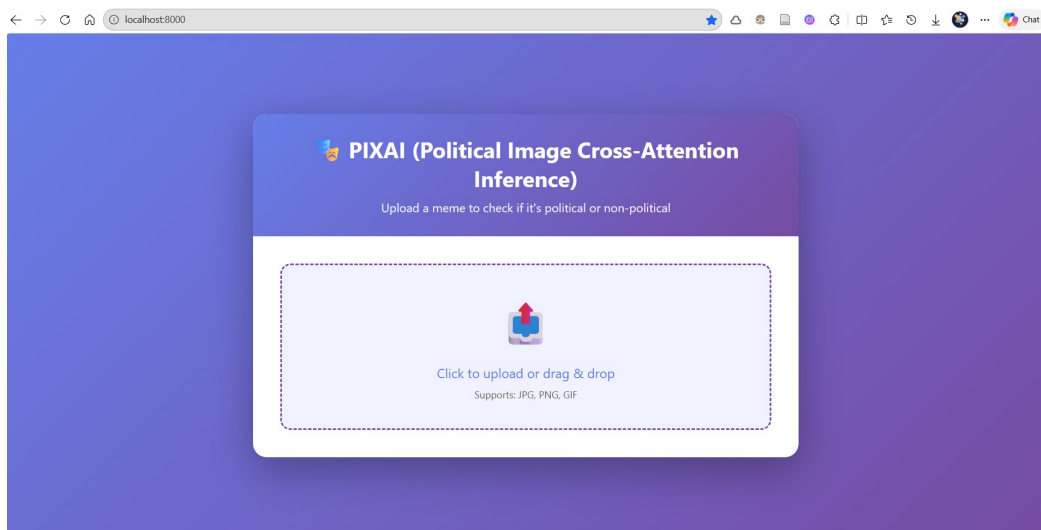


Figure 7.3: Webapp Interface



Analyze Meme

Upload Another

This meme is **Political**

97.9%

97.9%

Political

2.1%

Non-Political

Extracted Text

বাংলাদেশ

জাতীয়তাবাদী ছাত্রদল

আল্লাহ্ আকবার

লুসির তলে সংগ্রাম

চাঁদার টাকায় প্রোগ্রাম

Arariyan Memes

Normalized Text

বাংলাদেশ জাতীয়তাবাদী ছাত্রদল আল্লাহ্ আকবার লুসির তলে সংগ্রাম চাঁদার টাকায় প্রোগ্রাম

Figure 7.4: Correctly Classified Political Meme



Analyze Meme

Upload Another

This meme is **NonPolitical**

99.3%

0.7%

Political

99.3%

Non-Political

📄 Extracted Text

Snianix

MIST

SUST

JNU

imgflip.com

হাসচ-24

🌟 Normalized Text

Snianix MIST SUST JNU হাসচ 24

Figure 7.5: A Correctly Classified Non-political Meme



Figure 7.6: A Misclassified Political Meme due to Contextual Knowledge Limitation of the Trained Model. See Section 8.1 for detailed discussion.


```

(venv) D:\Desktop_Projects\pixai\python main.py
INFO: Will watch for changes in these directories: ['D:\Desktop_Projects\pixai']
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO: Started reloader process [20356] using WatchFiles
INFO: Started server process [18772]
INFO: Waiting for application startup.
Initializing Gemini text extractor...
Loading PIXAI model from model_task1.pth...
Using device: cpu
C:\Users\USER\AppData\Local\Programs\Python\Python311\Lib\site-packages\huggingface_hub\file_download.py:945: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use 'force_download=True'.
warnings.warn(
C:\Users\USER\AppData\Local\Programs\Python\Python311\Lib\site-packages\torch\utils.py:831: UserWarning: TypedStorage is deprecated. It will be removed in the future and UntypedStorage will be the only storage class. This should only matter to you if you are using storages directly. To access UntypedStorage directly, use tensor.untyped_storage() instead of tensor.storage().
  return self.fget._get_(instance, owner)()
✓ All models loaded successfully!
INFO: Application startup complete.
INFO: 127.0.0.1:4143 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:4143 - "GET /favicon.ico HTTP/1.1" 404 Not Found

```

Figure 7.7: Server Initialization Log

```

Making prediction...
INFO: 127.0.0.1:5465 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:3497 - "POST /predict HTTP/1.1" 200 OK
Extracting text with Gemini...
Cleaning and normalizing text...
Making prediction...
INFO: 127.0.0.1:14276 - "POST /predict HTTP/1.1" 200 OK
Extracting text with Gemini...
Cleaning and normalizing text...
Making prediction...
INFO: 127.0.0.1:8022 - "POST /predict HTTP/1.1" 200 OK
Extracting text with Gemini...
Cleaning and normalizing text...
Making prediction...
INFO: 127.0.0.1:8022 - "POST /predict HTTP/1.1" 200 OK
Extracting text with Gemini...
Cleaning and normalizing text...
Making prediction...
INFO: 127.0.0.1:4649 - "POST /predict HTTP/1.1" 200 OK
Extracting text with Gemini...
Cleaning and normalizing text...
Making prediction...
INFO: 127.0.0.1:11282 - "POST /predict HTTP/1.1" 200 OK
Extracting text with Gemini...
Cleaning and normalizing text...
Making prediction...
INFO: 127.0.0.1:7547 - "POST /predict HTTP/1.1" 200 OK

```

Figure 7.8: Server Log During Inference

Chapter 8

Discussion & Analysis

8.1 Interpretation of Findings

The experimental results demonstrate that the proposed PIXAI system is highly effective in identifying political intent within code-mixed Bengali memes. An overall accuracy of 93.24% and a Macro F1-score of 0.9159 indicate strong and balanced classification performance across both political and non-political classes.

From a real-world deployment perspective, the most significant outcome is the model’s high precision of 94.59% for the *Political* class, coupled with a very low False Positive rate, with only six misclassifications observed among more than 300 non-political samples. For automated content moderation systems, this characteristic is critical, as it minimizes the risk of incorrectly flagging benign content such as movie scenes, entertainment memes, or casual humor. Consequently, the likelihood of over-censorship and user dissatisfaction is substantially reduced.

However, the presence of 23 False Negatives, where political memes were classified as non-political, indicates that the model occasionally struggles with **subtle satire or deeply embedded political references rooted in cultural or contextual knowledge**. Such political contexts are continuously evolving, with new narratives, symbols, and references emerging over time, making static modeling inherently challenging. This suggests that while the system is conservative and suitable for safe deployment, incorporating a human-in-the-loop mechanism may be beneficial for reviewing borderline or ambiguous cases, and that dynamic modeling strategies may be required to adapt to the continuously evolving nature of political discourse.

8.2 Strengths of the Project

Robust Handling of Low Resource Code-Mixed Language (“Banglish”): Unlike traditional OCR-based pipelines that often fail when processing mixed-script or stylized text, the Vision–Language Model (VLM) based extraction module, implemented using Gemini/Qwen, successfully transcribes both Bengali and English text without introducing transliteration or segmentation errors. This enables the classifier to operate on linguistically accurate content rather than OCR-induced noise.

Context-Aware Cross-Attention Mechanism: The PIXAI architecture goes beyond simple feature concatenation by explicitly modeling interactions between visual and textual modalities through a dual attention framework. First, image representations query textual embeddings, enabling the model to focus on semantically relevant captions while ignoring background noise such as watermarks or decorative text. Second, a hybrid attention mechanism allows visual features to attend jointly to both textual and visual embeddings, capturing richer multimodal interactions and enhancing the model’s ability to interpret nuanced political intent in memes.

Production-Ready Asynchronous Architecture: Beyond serving as a research prototype, the system was deployed as a fully functional web application. The use of FastAPI with asynchronous request handling enables efficient management of network-bound VLM operations, maintaining responsiveness under concurrent user load—a capability often absent in academic implementations.

8.3 Limitations

Static Modeling in a Dynamic Political Landscape: The model is trained on a fixed snapshot of political discourse, which constrains its ability to generalize to newly emerging political narratives, symbols, and culturally grounded references. Given the continuously evolving nature of political expression in online spaces, particularly in meme culture, static training paradigms are inherently susceptible to concept drift over time.

Restricted Classification Scope: The system performs binary classification, distinguishing only between *Political* and *Non-Political* content. It does not account for finer-grained distinctions such as satire, hate speech, propaganda, or ideological stance, which limits its applicability for advanced social media analysis, policy enforcement, or forensic investigations.

Dependency on External APIs: The current deployed system relies on the Google Gemini API for multimodal text extraction, which introduces a dependency on stable internet connectivity and third-party service availability. This reliance may impact scalability, inference cost, and data privacy, particularly in sensitive content moderation scenarios. Additionally, changes in external API behavior or access policies could affect long-term system reliability.

Inference Latency and Computational Overhead: Despite architectural optimizations, the end-to-end inference pipeline incurs a latency of approximately 2–4 seconds per image. This delay primarily stems from the generative Vision–Language Model used for text extraction. While acceptable for interactive analysis or moderation workflows, this latency limits applicability in high-throughput environments such as real-time video streams or large-scale social media monitoring systems.

8.4 Recommendations for Future Development

Dynamic and Continual Learning Strategies: Future iterations of the system should incorporate dynamic modeling approaches, such as periodic fine-tuning, continual learning, or domain-adaptive training, to accommodate the rapidly evolving nature of political discourse. These strategies would enable the model to remain robust against emerging narratives, symbols, and culturally specific references that were absent during initial training.

Human-in-the-Loop and Reinforcement Learning: Introducing a human-in-the-loop mechanism for reviewing borderline or ambiguous cases can improve reliability while maintaining conservative deployment behavior. Feedback collected from such reviews can be leveraged within an active learning framework, allowing the model to iteratively update itself using high-value, real-world examples and mitigate false negatives caused by subtle political cues.

Localization of Vision–Language Model based Extraction: To reduce dependency on external services, future work should explore fine-tuning lightweight, open-source Vision–Language Models such as LLaVA-1.5 (7B) or Qwen-VL for on-premise deployment. Localized inference would improve latency, scalability, and data privacy, making the system more suitable for sensitive or large-scale applications.

Extension to Multi-Class or Multi-Label Classification: Expanding the PIXAI architecture to support multi-class or multi-label outputs would enable more nuanced content understanding. This includes differentiating between political intent, sentiment polarity, satire, misinformation, or propaganda, thereby enhancing the system’s utility for comprehensive content moderation and social media analytics.

Explainable AI (XAI) and Transparency of Prediction: Future enhancements will focus on visualizing the attention maps generated by the PIXAI cross-attention mechanism. By highlighting the specific image regions and textual tokens influencing predictions, the system can improve transparency, foster user trust, and provide valuable auditing tools for human moderators.

8.5 Reflection on Design & Implementation Decisions

Justification for VLM-Based Extraction over OCR: Initial experiments with Tesseract OCR produced unreliable results when applied to memes containing complex backgrounds and stylized fonts. Transitioning to a generative Vision–Language Model significantly improved text extraction quality, elevating the task from simple pattern recognition to meaningful semantic understanding.

Adoption of Cross-Attention Mechanism: Rather than relying solely on off-the-shelf multimodal embeddings, a custom cross-attention mechanism was implemented to improve interpretability. Attention weights provide insight into which textual tokens influence predictions for specific visual regions, increasing transparency and trust in the model’s decision-making process.

Choice of FastAPI for Backend Deployment: FastAPI was selected over tradi-

tional frameworks such as Flask due to its native support for asynchronous programming. The ability to leverage Python's `async/await` paradigm was essential for preventing server blocking during external API calls, thereby ensuring a responsive and scalable user experience.

Chapter 9

Life-Long Learning Impact

9.1 Technical & Professional Skills Acquired

The development of the PIXAI system served as a bridge between theoretical knowledge in deep learning and practical software engineering, enabling the acquisition of several long-term technical and professional competencies.

Multimodal System Architecture: The project provided hands-on experience in designing and integrating heterogeneous data streams from vision and language modalities. In particular, we learned how to align disparate embedding spaces by mapping CLIP-based visual representations with XGLM-based textual embeddings through a custom cross-attention mechanism. This competency is fundamental to modern multimodal AI systems.

Full-Stack Machine Learning Engineering: Beyond model development, the project emphasized end-to-end machine learning deployment. We gained experience in wrapping complex PyTorch models into production-ready APIs using FastAPI, managing multipart image uploads, validating request schemas with Pydantic, and handling asynchronous inference workflows.

Prompt Engineering: The project introduced us to prompt engineering as a critical skill for working with generative AI systems. By designing structured prompts (e.g., enforcing granular OCR-like extraction and valid JSON outputs), we learned to control the behavior of non-deterministic Vision–Language Models such as Google Gemini, ensuring consistent and reliable text extraction.

Resource-Constrained Optimization: We developed practical expertise in optimizing large models for limited hardware environments. Techniques such as 4-bit quantization (NF4) and gradient accumulation enabled the training and deployment of large language models on consumer-grade GPUs without significant performance degradation.

9.2 Learning New Technologies & Tools

The project required rapid adaptation to emerging technologies and methodologies that were not covered in the standard academic curriculum.

Hands on Generative VLMs: Initial reliance on Tesseract OCR proved inadequate for handling code-mixed and stylized meme text. This limitation necessitated a rapid tran-

sition to generative Vision–Language Models via the Google Generative AI SDK. The team learned to manage generation parameters such as temperature, token limits, and safety filters to accurately extract “Banglish” text where traditional OCR techniques failed.

Asynchronous Web Development (ASGI): Adapting from a synchronous programming background, we learned to develop asynchronous web services using FastAPI’s ASGI architecture. Mastery of `async` and `await` enabled efficient handling of non-blocking I/O operations, allowing the system to interact with external APIs while simultaneously serving multiple user requests.

Custom Architecture and Advanced Optimization Libraries: Rather than relying on off-the-shelf architectures, the project involved the design and implementation of a custom multimodal architecture tailored to the task requirements. Advanced optimization tools, including `bitsandbytes` for low-level model quantization and the MAD-GRAD optimizer for improved training stability, were employed. Effective use of these components required in-depth engagement with technical documentation and community resources, strengthening the ability to independently learn, adapt, and integrate emerging machine learning frameworks.

9.3 Future Growth & Directions

This project has shaped a clear trajectory for continued learning at the intersection of multimodal Natural Language Processing (NLP) and social science–driven analysis. As political communication increasingly manifests through images, text, and culturally grounded symbolism, future work in this domain requires models that move beyond unimodal linguistic understanding toward integrated, context-aware reasoning across modalities.

Building on the foundations established in this work, future learning will focus on advancing expertise in multimodal NLP, particularly in aligning visual semantics with linguistic structure and pragmatic meaning. This includes deeper exploration of cross-attention mechanisms, multimodal representation learning, and emerging Vision–Language architectures capable of modeling implicit context, satire, and sociocultural cues that are central to computational linguistics.

Finally, this project has reinforced the importance of continuous learning in a rapidly evolving research landscape. As multimodal data, social narratives, and AI methodologies continue to co-evolve, sustained engagement with both technical innovation and societal context will be necessary to develop systems that remain relevant, interpretable, and socially responsible.

Chapter 10

Conclusion

10.1 Summary of Work

This project successfully designed, implemented, and deployed **PIXAI** (Political Image Cross-Attention Inference), a novel multimodal deep learning system for the binary classification of political memes in the Bengali–English code-mixed domain.

Moving beyond the limitations of traditional unimodal classifiers and conventional OCR-based pipelines, the work introduced a two-stage processing framework. First, a Vision–Language Model (VLM) based text extraction module was engineered to perform semantic reading of noisy meme text. This approach effectively addressed the challenges posed by “Banglish” script mixing, noisy background and stylized typography commonly found in internet memes.

Second, a specialized neural architecture was developed to fuse visual and textual representations through a dual cross-attention mechanism. Visual embeddings extracted using CLIP query textual embeddings generated by XGLM, capturing semantic correspondence between image content and caption text. Visual embeddings also attend jointly to both textual and visual features as well, enabling the model to capture richer multimodal interactions and interpret political intent by explicitly conditioning textual semantics on visual context.

The complete system was finalized as a production-ready asynchronous FastAPI web application capable of real-time inference. Experimental evaluation demonstrated an overall classification accuracy of 93.24%, confirming that advanced multimodal reasoning can be effectively operationalized for practical content classification and moderation tasks.

10.2 Achievement of Objectives

The project was executed in strict alignment with the technical objectives outlined in Chapter 1.3. The successful achievement of each objective is summarized below.

Objective 1: Granular Multimodal Extraction

Status: Achieved.

Through the use of structured prompting with the Vision–Language Model, the system was able to perform fine-grained, OCR-like text extraction. This approach reliably distin-

guished semantically relevant caption text from background elements and accurately transcribed mixed-script Bengali–English content, avoiding the transcription errors and artifacts typically produced by conventional OCR engines.

Objective 2: Robust Textual Normalization

Status: Achieved.

A comprehensive text normalization pipeline was developed. Through NFKC Unicode normalization and regex-based artifact removal, including the elimination of URLs, emojis, and watermark text, the system ensured standardized and noise-free input for downstream language modeling.

Objective 3: Development of the PIXAI Architecture

Status: Achieved.

The custom PIXAI neural architecture was designed and implemented in PyTorch, incorporating a multi-head attention framework to align visual and textual modalities. The model employs two complementary attention mechanisms: one where visual features query textual embeddings, capturing semantic correspondence between image and caption, and another hybrid attention where image features attend to both textual and visual features, enabling richer multimodal interactions. This dual-attention design was successfully trained to fuse cross-modal information, resulting in effective semantic alignment and robust meme classification.

Objective 4: Real-Time Inference Deployment

Status: Achieved.

The trained model was deployed through an asynchronous FastAPI backend. A user-facing web interface was developed, allowing users to upload meme images and receive real-time predictions with confidence visualization for the *Political* and *Non-Political* classes.

10.3 Final Remarks

In an era where political discourse is increasingly shaped by visual and multimodal communication, the ability to automatically interpret the political intent embedded within internet memes is of critical importance. This project contributes meaningfully to the field of *Computational Sociolinguistics* by demonstrating that Bengali–English code-mixed content, often regarded as a low-resource and challenging domain, can be effectively analyzed through the integration of large Vision–Language Models and cross-attention mechanisms.

By prioritizing high precision and interpretability, PIXAI provides a transparent and reliable foundation for political content identification. The system represents a shift from surface-level keyword matching toward deeper semantic reasoning, offering a scalable and culturally aware approach for understanding the complex dynamics of modern political communication in digital spaces.

References

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [2] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” *CoRR*, vol. abs/1905.11946, 2019. [Online]. Available: <http://arxiv.org/abs/1905.11946>
- [3] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” *CoRR*, vol. abs/1911.02116, 2019. [Online]. Available: <http://arxiv.org/abs/1911.02116>
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [5] D. Kiela, H. Firooz, A. Mohan, V. Goswami, A. Singh, P. Ringshia, and D. Testuggine, “The hateful memes challenge: Detecting hate speech in multimodal memes,” *CoRR*, vol. abs/2005.04790, 2020. [Online]. Available: <https://arxiv.org/abs/2005.04790>
- [6] D. Dimitrov, F. Alam, M. Hasanain, A. Hasnat, F. Silvestri, P. Nakov, and G. Da San Martino, “SemEval-2024 task 4: Multilingual detection of persuasion techniques in memes,” in *Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024)*, A. K. Ojha, A. S. Doğruöz, H. Tayyar Madabushi, G. Da San Martino, S. Rosenthal, and A. Rosá, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 2009–2026. [Online]. Available: <https://aclanthology.org/2024.semeval-1.275/>
- [7] E. Hossain, O. Sharif, and M. M. Hoque, “MUTE: A multimodal dataset for detecting hateful memes,” in *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing: Student Research Workshop*, Y. Hanqi, Y. Zonghan, S. Ruder, and W. Xiaojun, Eds. Online: Association for Computational Linguistics, Nov. 2022, pp. 32–39. [Online]. Available: <https://aclanthology.org/2022.aacl-srw.5/>
- [8] M. Das and A. Mukherjee, “Banglaabusememe: A dataset for bengali abusive meme classification,” *arXiv preprint arXiv:2310.11748*, 2023.

- [9] S. Ahsan, E. Hossain, O. Sharif, A. Das, M. M. Hoque, and M. Dewan, “A multimodal framework to detect target aware aggression in memes,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 2487–2500.
- [10] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: pre-training of deep bidirectional transformers for language understanding,” *CoRR*, vol. abs/1810.04805, 2018. [Online]. Available: <http://arxiv.org/abs/1810.04805>
- [11] T. Hasan, A. Bhattacharjee, K. Samin, M. Hasan, M. Basak, M. S. Rahman, and R. Shahriyar, “Not low-resource anymore: Aligner ensembling, batch filtering, and new datasets for Bengali-English machine translation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, Nov. 2020, pp. 2612–2623. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-main.207>
- [12] Kaggle Competitions, “Poli-meme decode: Cuet cse fest,” <https://www.kaggle.com/competitions/poli-meme-decode-cuet-cse-fest>, 2025, accessed: 2025-01-02.
- [13] Q. Team, “Qwen2.5-vl,” January 2025. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5-vl/>
- [14] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, Y. Fan, K. Dang, M. Du, X. Ren, R. Men, D. Liu, C. Zhou, J. Zhou, and J. Lin, “Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution,” *arXiv preprint arXiv:2409.12191*, 2024.
- [15] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, “Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond,” *arXiv preprint arXiv:2308.12966*, 2023.
- [16] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” *CoRR*, vol. abs/2103.00020, 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [17] X. V. Lin, T. Mihaylov, M. Artetxe, T. Wang, S. Chen, D. Simig, M. Ott, N. Goyal, S. Bhosale, J. Du, R. Pasunuru, S. Shleifer, P. S. Koura, V. Chaudhary, B. O’Horo, J. Wang, L. Zettlemoyer, Z. Kozareva, M. T. Diab, V. Stoyanov, and X. Li, “Few-shot learning with multilingual language models,” *CoRR*, vol. abs/2112.10668, 2021. [Online]. Available: <https://arxiv.org/abs/2112.10668>
- [18] T. Hasan, A. Bhattacharjee, K. Samin, M. Hasan, M. Basak, M. S. Rahman, and R. Shahriyar, “Not low-resource anymore: Aligner ensembling, batch filtering, and new datasets for Bengali-English machine translation,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online:

Association for Computational Linguistics, Nov. 2020, pp. 2612–2623. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-main.207>

- [19] A. Defazio and S. Jelassi, “Adaptivity without compromise: A momentumized, adaptive, dual averaged gradient method for stochastic optimization,” 2021.

Appendix A

Training Dynamics Log

This appendix presents the detailed progression of the PIXAI model’s training over 5 epochs. The table A.1 below shows training and validation loss, accuracy, and Macro F1-score, demonstrating convergence behavior and highlighting the checkpoint used for testing.

Epoch	Train Loss	Train Acc (%)	Train Macro F1	Val Loss	Val Acc (%)	Val Macro F1
1	0.6038	79.87	0.7397	0.3797	90.21	0.8841
2	0.3124	91.41	0.8969	0.2605	94.64	0.9352
3	0.1954	94.96	0.9399	0.2125	94.64	0.9346
4	0.1311	95.95	0.9520	0.2220	95.10	0.9403
5	0.0939	97.40	0.9691	0.2351	94.64	0.9340

Table A.1: Training and Validation Metrics Across 5 Epochs

Epoch 4 yielded the best Validation Macro F1 and was used as the checkpoint for testing and deployment.

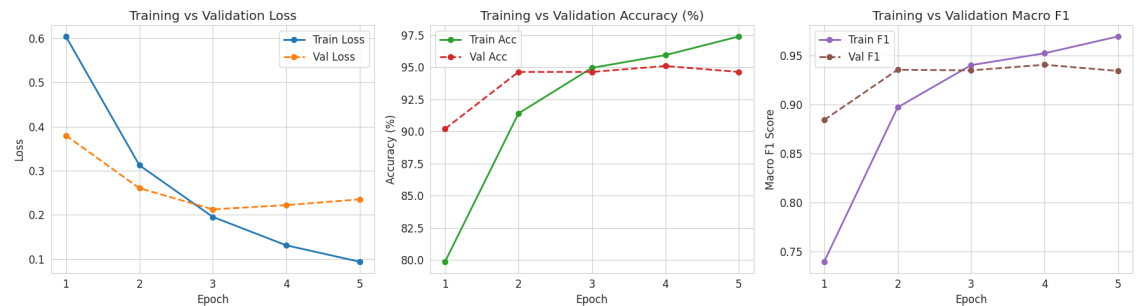


Figure A.1: (Training vs Validation) Loss, Accuracy and Macro F1

Figure A.1 illustrates the training dynamics of the PIXAI model across epochs, showing the progression of training and validation loss, accuracy, and Macro F1-score. The curves indicate stable convergence with a consistent reduction in training loss and strong alignment between training and validation performance. Notably, Epoch 4 achieves the highest validation Macro F1-score, confirming it as the optimal checkpoint and justifying its selection for final testing and deployment.